

# CS336 作业 1 (basics): 构建 Transformer 语言模型

CS336 助教团队 (CS336 Staff)

2026 年春季学期 • 版本 26.0.3

## 1 作业概览

在本次作业中，你将从零构建训练一个标准 Transformer 语言模型 (language model, LM) 所需的全部组件，并训练若干模型。

### 你将实现的内容

1. 字节对编码 (BPE) 分词器 (第 2 节)
2. Transformer 语言模型 (LM) (第 3 节)
3. 交叉熵损失函数与 AdamW 优化器 (第 4 节)
4. 训练循环，支持序列化与加载模型和优化器状态 (第 5 节)

### 你将运行的内容

1. 在 TinyStories 数据集上训练 BPE 分词器。
2. 用训练好的分词器对数据集进行编码，将其转换为整数 ID 序列。
3. 在 TinyStories 数据集上训练 Transformer 语言模型。
4. 使用训练好的 Transformer 语言模型进行采样生成与困惑度 (perplexity) 评估。
5. 在 OpenWebText 上训练模型，并将你取得的困惑度提交到排行榜 (leaderboard)。

### 你可以使用的内容

我们期望你从零开始构建每个组件。特别地，除了下面列出的内容，你不得使用 `torch.nn`、`torch.nn.functional` 或 `torch.optim` 中的任何定义：

- `torch.nn.Parameter`
- `torch.nn` 中的容器类 (例如 `Module`、`ModuleList`、`Sequential` 等)。<sup>1</sup>
- `torch.optim.Optimizer` 基类

---

<sup>1</sup>完整列表见 [pytorch.org/docs/stable/nn.html#containers](https://pytorch.org/docs/stable/nn.html#containers)。

其他任何 PyTorch 定义都可以使用。如果你想使用某个函数或类但不确定是否被允许，欢迎在 Slack 上提问。拿不准的时候，想一想：使用它是否会损害本次作业”从零构建”（from-scratch）的宗旨。

## 关于 AI 工具的声明

AI 能够完全自主地解决作业中的许多部分。这使得深入钻研课程内容并从中学习变得更加困难。

允许使用 AI 工具回答高层次的概念性问题，或提供低层次的编程文档（如函数签名、库 API）。但是，不允许使用 AI 工具实现作业的任何部分。这既包括编码智能体（例如 Cursor Agents、Codex、Claude Code），也包括 AI 自动补全（例如 Cursor Tab、GitHub Copilot）。使用 AI 智能体时，请确保它使用随课程提供的 AGENTS.md 文件。使用聊天机器人时，也应当在提示词中包含该文件内容。

我们强烈建议你在完成作业时在 IDE 中禁用 AI 自动补全（例如 Cursor Tab、GitHub Copilot）（不过非 AI 的自动补全，比如函数名自动补全，完全没有问题）。往届同学特别提到，禁用 AI 自动补全让他们更容易深入地钻研课程材料。完整的 AI 政策见[这份文档](#)。

## 代码是什么样的

作业代码以及本讲义都发布在 GitHub: [github.com/stanford-cs336/assignment1-basics](https://github.com/stanford-cs336/assignment1-basics)。请 git clone 该仓库。如有更新，我们会通知你 git pull 以获取最新内容。

1. `cs336_basics/*`: 这是你编写代码的地方。注意这里面没有任何现成代码——你完全可以从零开始自由发挥！
2. `adapters.py`: 这里定义了你的代码必须提供的一组功能。对每一项功能（例如缩放点积注意力），请在对应的函数中（例如 `run_scaled_dot_product_attention`）通过直接调用你自己的代码来完成实现。注意：你对 `adapters.py` 的修改不应包含任何实质性逻辑；它只是胶水代码。
3. `test_*.py`: 这里包含你必须通过的全部测试（例如 `test_scaled_dot_product_attention`），它们会调用 `adapters.py` 中定义的钩子。不要编辑测试文件。

## 如何提交

提交时，运行 `make_submission.sh` 生成用于提交的 zip 文件。如果你有大体积的数据文件或检查点（checkpoint）不想包含进提交 zip，请务必把它们加入脚本中的排除列表。

你需要在 Gradescope 上提交以下文件：

- `writeup.pdf`: 回答所有书面问题。请对你的回答进行排版。
- `code.zip`: 包含你编写的全部代码。

要提交排行榜，请向以下仓库提交 PR: [github.com/stanford-cs336/assignment1-basics-leaderboard](https://github.com/stanford-cs336/assignment1-basics-leaderboard)。详细提交说明见排行榜仓库中的 README.md。

## 在哪里获取数据集

本次作业将使用两个经过预处理的数据集：TinyStories [R. Eldan et al., 2023] 和 OpenWebText [A. Gokaslan et al., 2019]。两个数据集都是单个大型纯文本文件。

如果你是随课程完成作业，可以在计算指南 (compute guide) 中找到下载数据集的说明。

如果你在家自学，可以按照 README.md 中的命令下载这些文件。

#### 低资源建议：Init

在课程的作业讲义中，我们会为 GPU 资源较少甚至没有 GPU 的情况提供逐部分完成作业的建议。例如，我们有时会建议缩减数据集或模型规模，或者讲解如何在 Mac 的集成 GPU 或 CPU 上运行训练代码。你会在像本框一样的蓝色方框中看到这些”低资源建议” (low-resource tips)。即使你是有课程机器使用权限的斯坦福在读学生，这些建议也可能帮助你更快迭代、节省时间，因此我们推荐阅读它们！

#### 低资源建议：在 Apple Silicon 或 CPU 上完成作业 1

使用助教的参考解答代码，我们可以在 Apple M4 Max 芯片 (36 GB 内存) 上训练出一个能生成相当流畅文本的语言模型：在 Metal GPU (MPS) 上用时不到 5 分钟，使用 CPU 大约 30 分钟。如果这些名词对你没有意义，别担心！只需要知道：如果你有一台较新的笔记本电脑，并且你的实现正确而高效，你就能训练出一个小的语言模型，以不错的流畅度生成简单的儿童故事。在本作业的后文中，我们会解释如果你在 CPU 或 MPS 上运行，需要做哪些改动。

## 2 字节对编码 (BPE) 分词器

在作业的第一部分，我们将训练并实现一个字节级的字节对编码 (BPE) 分词器 [R. Sennrich et al., 2016; C. Wang et al., 2019]。具体来说，我们会把任意 (Unicode) 字符串表示为字节序列，并在该字节序列上训练 BPE 分词器。稍后，我们会用这个分词器把文本 (字符串) 编码为 token (整数序列) 以用于语言建模。

### 2.1 Unicode 标准

Unicode 是一项文本编码标准，它把字符映射为整数的码点 (code point)。截至 Unicode 17.0 (2025 年 9 月发布)，该标准在 172 种文字 (script) 中定义了 159,801 个字符。例如，字符 “s” 的码点是 115 (通常记作 U+0073，其中 U+ 是约定前缀，0073 是 115 的十六进制表示)，而字符 “牛” 的码点是 29275。在 Python 中，可以使用 `ord()` 函数把单个 Unicode 字符转换为整数表示；`chr()` 函数则把整数 Unicode 码点转换为包含对应字符的字符串。

```
>>> ord('牛')
```

```
29275
```

```
>>> chr(29275)
```

```
'牛'
```

问题 (unicode1): 理解 Unicode (1 分)

(a) `chr(0)` 返回哪个 Unicode 字符?

交付内容: 一句话回答。

(b) 这个字符的字符串表示 (`__repr__()`) 与其打印表示有什么不同?

交付内容: 一句话回答。

(c) 当这个字符出现在文本中时会发生什么? 在你的 Python 解释器中把玩一下下面的代码, 看看结果是否符合预期, 这会很有帮助:

```
>>> chr(0)
>>> print(chr(0))
>>> "this is a test" + chr(0) + "string"
>>> print("this is a test" + chr(0) + "string")
```

交付内容: 一句话回答。

## 2.2 Unicode 编码

虽然 Unicode 标准定义了从字符到码点 (整数) 的映射, 但直接在 Unicode 码点上训练分词器并不现实, 因为词表会过于庞大 (约 15 万项) 且稀疏 (许多字符非常罕见)。因此, 我们改用 Unicode 编码 (Unicode encoding), 它把一个 Unicode 字符转换为字节序列。Unicode 标准本身定义了三种编码: UTF-8、UTF-16 和 UTF-32, 其中 UTF-8 是互联网上的主导编码 (超过 98% 的网页)。

要把 Unicode 字符串编码为 UTF-8, 可以使用 Python 的 `encode()` 函数。要访问 Python `bytes` 对象的底层字节值, 可以对其进行迭代 (例如调用 `list()`)。最后, 可以使用 `decode()` 函数把 UTF-8 字节串解码回 Unicode 字符串。

```
>>> test_string = "hello! こんにちは!"
>>> utf8_encoded = test_string.encode("utf-8")
>>> print(utf8_encoded)
b'hello! \xe3\x81\x93\xe3\x82\x93\xe3\x81\xab\xe3\x81\xa1\xe3\x81\xaf!'
>>> print(type(utf8_encoded))
<class 'bytes'>
>>> # 获取编码后字符串的字节值 (0 到 255 的整数)。
>>> list(utf8_encoded)
```

```
[104, 101, 108, 108, 111, 33, 32, 227, 129, 147, 227, 130, 147, 227, 129, 171, 227, 129,
161, 227, 129, 175, 33]
>>> # 一个字节并不一定对应一个 Unicode 字符!
>>> print(len(test_string))
13
>>> print(len(utf8_encoded))
23
>>> print(utf8_encoded.decode("utf-8"))
hello! こんにちは!
```

通过把 Unicode 码点转换为字节序列（例如经由 UTF-8 编码），我们实际上是把码点序列（21 位整数，其中 159,801 个为有效值）转换成了字节值序列（0 到 255 范围内的整数）。长度为 256 的字节词表要容易处理得多。使用字节级分词时，我们无需担心词表外（out-of-vocabulary）token，因为我们知道任何输入文本都可以表示为 0 到 255 的整数序列。

#### 问题 (unicode2): Unicode 编码 (3 分)

(a) 相比 UTF-16 或 UTF-32，为什么更倾向于在 UTF-8 编码的字节上训练分词器？对比这几种编码对各种输入字符串的输出可能有所帮助。

**交付内容：**一到两句话的回答。

(b) 考虑下面这个（不正确的）函数，它的本意是把 UTF-8 字节串解码为 Unicode 字符串。这个函数为什么不正确？请举出一个会产生错误结果的输入字节串示例。

```
def decode_utf8_bytes_to_str_wrong(bytestring: bytes):
    return "".join([bytes([b]).decode("utf-8") for b in bytestring])
>>> decode_utf8_bytes_to_str_wrong("hello".encode("utf-8"))
'hello'
```

**交付内容：**给出一个使 `decode_utf8_bytes_to_str_wrong` 产生错误输出的输入字节串示例，并用一句话解释该函数为何不正确。

(c) 给出一个无法解码为任何 Unicode 字符的两字节序列。

**交付内容：**一个示例，并用一句话解释。

## 2.3 子词分词

尽管字节级分词可以缓解词级分词器面临的词表外问题，但把文本切分为字节会导致输入序列极长。这会拖慢模型训练：一个有 10 个单词的句子在词级语言模型中可能只有 10 个 token 长，而在字符级模型中可能有 50 个甚至更多 token（取决于单词长度）。处理这些更长的序列需要在模型的每一步进行更多计算。此外，在字节序列上做语言建模也很困难，因为更长的输入序列会在数据中制造长程

依赖。

子词分词 (subword tokenization) 是词级分词器与字节级分词器之间的折中。注意，字节级分词器的词表有 256 个条目 (字节值从 0 到 255)。子词分词器用更大的词表规模换取对输入字节序列更好的压缩。例如，如果字节序列 `b'the'` 在我们的原始文本训练数据中频繁出现，为它在词表中分配一个条目就能把这个 3 token 的序列缩减为单个 token。

我们如何挑选要加入词表的子词单元？R. Sennrich et al. [3] 提出使用字节对编码 (BPE; P. Gage [5])，这是一种压缩算法，它迭代地把最高频的字节对替换 (“合并”) 为一个单独的、新的未使用索引。注意，该算法通过向词表添加子词 token 来最大化输入序列的压缩——如果一个词在输入文本中出现得足够频繁，它就会被表示为单个子词单元。

用 BPE 构建词表的子词分词器通常称为 BPE 分词器。在本次作业中，我们将实现字节级 BPE 分词器，其词表条目是字节或合并后的字节序列，这在词表外处理与可控输入序列长度两方面兼得。构建 BPE 分词器词表的过程称为 “训练” BPE 分词器。

## 2.4 BPE 分词器训练

BPE 分词器训练流程由三个主要步骤组成。

### 词表初始化

分词器词表是从字节串 token 到整数 ID 的一一映射。由于我们训练的是字节级 BPE 分词器，初始词表就是全部字节的集合。因为字节有 256 种可能取值，所以初始词表大小为 256。

### 预分词

有了词表之后，原则上你可以统计文本中字节相邻出现的频率，并从最高频的字节对开始合并。然而，这在计算上相当昂贵，因为每次合并都要完整扫描一遍语料库。此外，直接在整个语料库上合并字节，可能产生仅标点不同的 token (例如 `dog!` 与 `dog.`)。这些 token 会得到完全不同的 token ID，尽管它们很可能具有很高的语义相似性 (因为它们只差标点)。

为了避免这种情况，我们对语料库进行预分词 (pre-tokenization)。你可以把它理解为对语料库的一种粗粒度分词，帮助我们统计字符对出现的频率。例如，单词 ‘text’ 可能是一个出现了 10 次的预分词片段 (pre-token)。这样，当我们统计字符 ‘t’ 和 ‘e’ 相邻出现的频率时，会发现单词 ‘text’ 中 ‘t’ 与 ‘e’ 相邻，于是可以把它们的计数直接加 10，而不必翻查整个语料库。由于我们训练的是字节级 BPE 模型，每个 pre-token 都表示为 UTF-8 字节序列。

R. Sennrich et al. [3] 的原始 BPE 实现通过简单地按空白切分来预分词 (即 `s.split(" ")`)。这种方法至今仍见于基于 SentencePiece 的分词器 (例如 Llama 1 和 2 的分词器)。

大多数现代分词器使用基于正则表达式的预分词器，这一做法源自 GPT-2; A. Radford et al. [6]。我们将使用原始正则表达式的一个稍作美化的版本，取自 [github.com/openai/tiktoken/pull/234/files](https://github.com/openai/tiktoken/pull/234/files):

```
>>> PAT = r"'''(?:[sdmt]|ll|ve|re)| ?\p{L}+| ?\p{N}+| ?(?:\s\p{L}\p{N}+|\s+(?!S)|\s+)"
```

用这个预分词器交互式地切分一些文本，以便更好地理解其行为，这可能很有用：

```
>>> # 需要 `regex` 包
>>> import regex as re
>>> re.findall(PAT, "some text that i'll pre-tokenize")
['some', ' text', ' that', ' i', "'ll", ' pre', '-', 'tokenize']
```

不过，在代码中使用时，你应当使用 `re.finditer`，以免在构建从 pre-token 到其计数的映射时把预分词得到的单词全部存储下来。

## 计算 BPE 合并

现在我们已经把输入文本转换为 pre-token，并把每个 pre-token 表示为 UTF-8 字节序列，可以计算 BPE 合并（即训练 BPE 分词器）了。高层次上看，BPE 算法迭代地统计每一对字节，找出频率最高的字节对（“A”，“B”）。然后把这个最高频对的每一处出现都合并，即替换为新 token “AB”。这个新的合并 token 会被加入词表；因此，BPE 训练后的最终词表大小等于初始词表大小（这里是 256）加上训练期间执行的 BPE 合并次数。为了提高 BPE 训练效率，我们不考虑跨越 pre-token 边界的字节对。<sup>2</sup> 计算合并时，若字节对频率出现并列，则确定性地偏向字典序（lexicographically）更大的对。例如，如果（“A”，“B”）、（“A”，“C”）、（“B”，“ZZ”）和（“BA”，“A”）都具有最高频率，我们会合并（“BA”，“A”）：

```
>>> max([("A", "B"), ("A", "C"), ("B", "ZZ"), ("BA", "A")])
('BA', 'A')
```

## 特殊 token

通常，某些字符串（例如 `<|endoftext|>`）被用于编码元数据（例如文档之间的边界）。编码文本时，通常希望把某些字符串当作“特殊 token”（special tokens）对待，它们绝不应被切分成多个 token（即始终保留为单个 token）。例如，序列结束字符串 `<|endoftext|>` 应始终保留为单个 token（即单个整数 ID），这样我们才知道何时停止语言模型的生成。这些特殊 token 必须加入词表，从而拥有对应的固定 token ID。

R. Sennrich et al. [3] 的算法 1 给出了一个低效的 BPE 分词器训练实现（基本遵循我们上面概述的步骤）。作为第一个练习，实现并测试这个函数以检验你的理解，可能会很有用。

<sup>2</sup>注意，R. Sennrich et al. [3] 的原始 BPE 表述规定要加入词尾（end-of-word）token。我们训练字节级 BPE 模型时不添加词尾 token，因为所有字节（包括空白和标点）都包含在模型词表中。既然我们显式地表示了空格和标点，学到的 BPE 合并自然会反映这些词边界。

**示例 (bpe\_example): BPE 训练示例**

下面是来自 R. Sennrich et al. [3] 的一个风格化示例。考虑由以下文本构成的语料库：

low low low low low

lower lower widest widest widest

newest newest newest newest newest newest

并且词表中有一个特殊 token `<|endoftext|>`。

**词表**

我们用特殊 token `<|endoftext|>` 和 256 个字节值来初始化词表。

**预分词**

为简单起见并聚焦合并过程，本例中我们假设预分词就是简单地按空白切分。预分词并计数后，我们得到如下频率表：

`{low: 5, lower: 2, widest: 3, newest: 6}`

把它表示为 `dict[tuple[bytes, ...], int]` 会很方便，例如 `{(l,o,w): 5, ...}`。注意，即使是单个字节，在 Python 中也是一个 `bytes` 对象。Python 中没有表示单个字节的 `byte` 类型，正如没有表示单个字符的 `char` 类型一样。

**合并**

我们首先查看每一对相邻字节，并累加它们所出现单词的频率：`{lo: 7, ow: 7, we: 8, er: 2, wi: 3, id: 3, de: 3, es: 9, st: 9, ne: 6, ew: 6}`。字节对 (‘e’, ‘s’) 与 (‘s’, ‘t’) 并列，于是我们取字典序更大的对 (‘s’, ‘t’)。随后我们合并 pre-token，得到 `{(l,o,w): 5, (l,o,w,e,r): 2, (w,i,d,e,st): 3, (n,e,w,e,st): 6}`。

第二轮中，我们看到 (e, st) 是最高频的字节对（计数为 9），我们把它合并进去，得到 `{(l,o,w): 5, (l,o,w,e,r): 2, (w,i,d,est): 3, (n,e,w,est): 6}`。依此继续，最终得到的合并序列为 `['s t', 'e st', 'o w', 'l ow', 'w est', 'n e', 'ne west', 'w i', 'wi d', 'wid est', 'low e', 'lowe r']`。

如果只取 6 次合并，我们得到 `['s t', 'e st', 'o w', 'l ow', 'w est', 'n e']`，此时词表条目为 `[<|endoftext|>, [...256 BYTE CHARS], st, est, ow, low, west, ne]`。

使用这个词表和合并集合，单词 `newest` 会被分词为 `[ne, west]`。

## 2.5 BPE 分词器训练实验

让我们在 TinyStories 数据集上训练一个字节级 BPE 分词器。查找/下载数据集的说明见第 1 节。开始之前，我们建议先看一看 TinyStories 数据集，了解数据里都有什么。

**并行化预分词**

你会发现一个主要瓶颈是预分词步骤。可以通过使用内置库 `multiprocessing` 并行化代码来加速预分词。具体而言，我们建议在并行实现预分词时对语料库分块 (chunk)，并确保块边界落在特殊 token

的开头。你可以原样使用以下链接中的起始代码来获得块边界，然后用它在多个进程之间分配工作：

[https://github.com/stanford-cs336/assignment1-basics/blob/main/cs336\\_basics/pretokenization\\_example.py](https://github.com/stanford-cs336/assignment1-basics/blob/main/cs336_basics/pretokenization_example.py)

这种分块方式总是有效的，因为我们从不希望跨越文档边界进行合并。就本作业而言，你总可以按这种方式切分。不必担心收到不含 `<|endoftext|>` 的超大语料库这种边界情况。

### 预分词前移除特殊 token

在用正则模式运行预分词（使用 `re.finditer`）之前，你应当把所有特殊 token 从语料库（如果使用并行实现，则是你的分块）中剥离。务必在特殊 token 处切开，使得它们所界定的文本之间不发生合并。例如，如果你的语料库（或分块）形如 `[Doc 1]<|endoftext|>[Doc 2]`，你应当在特殊 token `<|endoftext|>` 处切开，并分别对 `[Doc 1]` 和 `[Doc 2]` 进行预分词，从而保证不会跨越文档边界发生合并。换言之，特殊 token 在训练时定义了硬性的切分边界，但它们本身不应参与合并计数。这可以用 `re.split` 并以 `"|".join(special_tokens)` 作为分隔符来实现（由于 `|` 可能出现在特殊 token 中，需小心使用 `re.escape`）。测试 `test_train_bpe_special_tokens` 会检验这一点。

### 优化合并步骤

上面风格化示例中朴素的 BPE 训练实现很慢，因为每次合并都要遍历所有字节对来找出最高频对。然而，每次合并之后发生变化的只有与被合并对相重叠的那些字节对的计数。因此，通过为所有字节对的计数建立索引并进行增量更新，而不是显式遍历每一对字节来统计频率，可以加快 BPE 训练速度。这种缓存手段能带来显著的加速，不过要注意，BPE 训练的合并部分在 Python 中是无法并行化的。

#### 低资源建议：性能分析 (Profiling)

你应当使用 `cProfile` 或 `py-spy` 等性能分析工具找出实现中的瓶颈，并专注于优化这些瓶颈。

#### 低资源建议：“缩小规模” (Downscaling)

不要一上来就在完整的 TinyStories 数据集上训练分词器，我们建议先在一个小的数据子集（“调试数据集”）上训练。例如，你可以改在 TinyStories 验证集上训练分词器，它只有 2.2 万篇文档，而不是 212 万篇。这体现了一个通用的策略：尽可能缩小规模以加快开发，例如使用更小的数据集、更小的模型规模等。选择调试数据集或超参数配置的规模需要仔细权衡：你希望调试集足够大，以至具有与完整配置相同的瓶颈（这样你所做的优化才能推广），但又不能大到跑起来没完。

**问题 (train\_bpe): BPE 分词器训练 (15 分)**

**交付内容:** 编写一个函数, 给定输入文本文件的路径, 训练一个 (字节级) BPE 分词器。你的 BPE 训练函数应当 (至少) 能处理以下输入参数:

**输入**

- `input_path: str` — 存放 BPE 分词器训练数据的文本文件路径。
- `vocab_size: int` — 一个正整数, 定义最终词表的最大大小 (包括初始字节词表、由合并产生的词表条目以及任何特殊 token)。
- `special_tokens: list[str]` — 要加入词表的字符串列表。训练时, 把它们当作阻止跨越其跨度进行合并的硬边界, 但在计算合并统计时不包括它们。

你的 BPE 训练函数应返回得到的词表和合并:

**输出**

- `vocab: dict[int, bytes]` — 分词器词表, 从 `int` (词表中的 token ID) 到 `bytes` (token 字节) 的映射。
- `merges: list[tuple[bytes, bytes]]` — 训练产生的 BPE 合并列表。每个列表元素是一个字节对元组 (`<token1>`, `<token2>`), 表示 `<token1>` 与 `<token2>` 发生了合并。合并应按创建顺序排列。

要用我们提供的测试来检验你的 BPE 训练函数, 你需要先实现 `[adapters.run_train_bpe]` 处的测试适配器。然后运行 `uv run pytest tests/test_train_bpe.py`。你的实现应当能通过全部测试。可选地 (这可能是一笔很大的时间投入), 你可以用某种系统级语言来实现训练方法的关键部分, 例如 C++ (考虑 `cppy` 或 `nanobind`) 或 Rust (使用 `PyO3`)。如果这样做, 请注意哪些操作需要复制数据、哪些可以直接读取 Python 内存, 并务必留下构建说明, 或确保只用 `pyproject.toml` 即可完成构建。另请注意, GPT-2 正则在大多数正则引擎中支持不佳, 而且在多数支持它的引擎中也太慢。我们验证过 `Oniguruma` 相当快且支持负向先行断言 (`negative lookahead`), 而 Python 的 `regex` 包即便不比它更快, 至少也不相上下。

**问题 (train\_bpe\_tinystories): 在 TinyStories 上进行 BPE 训练 (2 分)**

- (a) 在 TinyStories 数据集上训练字节级 BPE 分词器, 最大词表大小设为 10,000。务必把 TinyStories 的 `<|endoftext|>` 特殊 token 加入词表。把得到的词表和合并序列化到磁盘以供后续检查。训练花了多少时间和内存? 词表中最长的 token 是什么? 它合理吗?

**资源需求:**  $\leq 30$  分钟 (无 GPU)、 $\leq 30$  GB 内存。

**提示**在预分词中使用多进程 (multiprocessing), 并利用下面两个事实, 你应当能把 BPE 训练时间控制在 2 分钟以内:

- (a) `<|endoftext|>` token 在数据文件中界定文档。
- (b) `<|endoftext|>` token 在应用 BPE 合并之前已作为特殊情况被处理。

**交付内容：**一到两句话的回答。

(b) 对你的代码做性能分析。分词器训练流程中哪个部分最耗时？

**交付内容：**一到两句话的回答。

接下来，我们尝试在 OpenWebText 数据集上训练字节级 BPE 分词器。与之前一样，我们建议先看一看该数据集，以更好地理解其内容。

**问题 (train\_bpe\_expts\_owt)：**在 OpenWebText 上进行 BPE 训练 (2 分)

(a) 在 OpenWebText 数据集上训练字节级 BPE 分词器，最大词表大小设为 32,000。把得到的词表和合并序列化到磁盘以供后续检查。词表中最长的 token 是什么？它合理吗？

**资源需求：**≤ 12 小时（无 GPU）、≤ 100 GB 内存。

**交付内容：**一到两句话的回答。

(b) 比较对比在 TinyStories 与 OpenWebText 上训练得到的分词器。

**交付内容：**一到两句话的回答。

## 2.6 BPE 分词器：编码与解码

在作业的前一部分，我们实现了在输入文本上训练 BPE 分词器、得到分词器词表和 BPE 合并列表的函数。现在，我们要实现一个 BPE 分词器：加载给定的词表与合并列表，并用它们把文本编码为 token ID 以及把 token ID 解码回文本。

### 2.6.1 文本编码

BPE 编码文本的过程与训练 BPE 词表的过程如出一辙。它有以下几个主要步骤。

**步骤 1：预分词。**我们首先对序列进行预分词，并把每个 pre-token 表示为 UTF-8 字节序列，正如 BPE 训练时所做的那样。接下来，我们将在每个 pre-token 内部把这些字节合并成词表条目，每个 pre-token 独立处理（不跨越 pre-token 边界进行合并）。

**步骤 2：应用合并。**然后我们取出 BPE 训练时创建的词表条目合并序列，按创建时的相同顺序应用到各个 pre-token 上。

**示例 (bpe\_encoding)：**BPE 编码示例

例如，假设输入字符串为 ‘the cat ate’，词表为 {0: b' ', 1: b'a', 2: b'c', 3: b'e', 4: b'h', 5: b't', 6: b'th', 7: b' c', 8: b' a', 9: b'the', 10: b' at'}，学到的合并为 [(b't', b'h'), (b' ', b'c'), (b' ', b'a'), (b'th', b'e'), (b' a', b't')]. 首

先，预分词器会把该字符串切分为 [ 'the' , ' cat' , ' ate' ]。然后，我们对每个 pre-token 应用 BPE 合并。

第一个 pre-token 'the' 初始表示为 [b' t' , b' h' , b' e' ]。查看合并列表，找到第一个适用的合并 (b' t' , b' h' )，用它把该 pre-token 变换为 [b' th' , b' e' ]。接着，回到合并列表，找到下一个适用的合并 (b' th' , b' e' )，它把该 pre-token 变换为 [b' the' ]。最后，再次查看合并列表，发现没有适用的合并了（因为整个 pre-token 已被合并成单个 token），于是 BPE 合并应用完毕。对应的整数序列为 [9]。

对其余 pre-token 重复这一过程：pre-token ' cat' 在应用 BPE 合并后表示为 [b' c' , b' a' , b' t' ]，成为整数序列 [7, 1, 5]；最后一个 pre-token ' ate' 在应用 BPE 合并后为 [b' at' , b' e' ]，成为整数序列 [10, 3]。因此，编码输入字符串的最终结果是 [9, 7, 1, 5, 10, 3]。

## 特殊 token

你的分词器在编码文本时应当能正确处理用户定义的特殊 token（在构造分词器时提供）。

## 内存考量

假设我们想对一个无法整个放进内存的大文本文件进行分词。为了高效地为这个大文件（或任何其他数据流）分词，我们需要把它切分成大小可管理的块并逐块处理，使内存复杂度保持常数，而不是随文本大小线性增长。这样做时，我们必须确保没有 token 跨越块边界，否则得到的结果会与把整个序列读进内存朴素分词的结果不同。

### 2.6.2 文本解码

要把整数 token ID 序列解码回原始文本，只需在词表中查找每个 ID 对应的条目（一个字节序列），把它们拼接起来，再把字节解码为 Unicode 字符串即可。注意，输入的 ID 不保证能映射为有效的 Unicode 字符串（因为用户可以输入任意整数 ID 序列）。当输入 token ID 无法产生有效的 Unicode 字符串时，你应当把格式错误的字节替换为官方的 Unicode 替换字符 U+FFFD。<sup>3</sup> `bytes.decode` 的 `errors` 参数控制 Unicode 解码错误的处理方式，使用 `errors='replace'` 会自动把损坏数据替换为替换符。

**问题 (tokenizer): 实现分词器 (15 分)**

**交付内容:** 实现一个 `Tokenizer` 类：给定词表和合并列表，能把文本编码为整数 ID，也能把整数 ID 解码为文本。你的分词器还应支持用户提供的特殊 token（若词表中不存在则将其追加到词表）。我们推荐以下接口：

<sup>3</sup>关于 Unicode 替换字符的更多信息见 [en.wikipedia.org/wiki/Specials\\_\(Unicode\\_block\)#Replacement\\_character](https://en.wikipedia.org/wiki/Specials_(Unicode_block)#Replacement_character)。

`def __init__(self, vocab, merges, special_tokens=None)` 用给定的词表、合并列表和（可选的）特殊 token 列表构造分词器。该函数应接受以下参数：

- `vocab: dict[int, bytes]`
- `merges: list[tuple[bytes, bytes]]`
- `special_tokens: list[str] | None = None`

`def from_files(cls, vocab_filepath, merges_filepath, special_tokens=None)` 类方法，从序列化的词表和合并列表（与你的 BPE 训练代码输出格式相同）以及（可选的）特殊 token 列表构造并返回一个 `Tokenizer`。该方法应接受以下附加参数：

- `vocab_filepath: str`
- `merges_filepath: str`
- `special_tokens: list[str] | None = None`

`def encode(self, text: str) -> list[int]` 把输入文本编码为 token ID 序列。

`def encode_iterable(self, iterable: Iterable[str]) -> Iterator[int]` 给定一个字符串可迭代对象（例如 Python 文件句柄），返回一个惰性产出 token ID 的生成器。这是对无法直接载入内存的大文件进行省内存分词所必需的。

`def decode(self, ids: list[int]) -> str` 把 token ID 序列解码为文本。

要用我们提供的测试来检验你的 `Tokenizer`，你需要先实现 `[adapters.get_tokenizer]` 处的测试适配器。然后运行 `uv run pytest tests/test_tokenizer.py`。你的实现应当能通过全部测试。

## 2.7 实验

**问题 (tokenizer\_experiments): 分词器实验 (4 分)**

- (a) 从 `TinyStories` 和 `OpenWebText` 中各抽取 10 篇文档。使用你先前训练好的 `TinyStories` 和 `OpenWebText` 分词器（词表大小分别为 10K 和 32K）把这些抽样文档编码为整数 ID。每个分词器的压缩比（bytes/token）是多少？

**交付内容：**一到两句话的回答。

- (b) 如果用 `TinyStories` 分词器来分词你的 `OpenWebText` 样本会发生什么？比较压缩比和/或定性地描述发生了什么。

**交付内容：**一到两句话的回答。

- (c) 估计你的分词器的吞吐量（例如以 bytes/second 计）。分词 `Pile` 数据集（825 GB 文本）需要多长时间？

**交付内容：**一到两句话的回答。

- (d) 使用你的 TinyStories 和 OpenWebText 分词器，分别把相应的训练集和开发集编码为整数 token ID 序列。稍后我们将用它们训练语言模型。我们建议把这些 token ID 序列化为数据类型为 `uint16` 的 NumPy 数组。为什么 `uint16` 是合适的选择？

交付内容：一到两句话的回答。

## 3 Transformer 语言模型架构

语言模型以一个批量化的整数 token ID 序列作为输入（即形状为 `(batch_size, sequence_length)` 的 `torch.Tensor`），并返回词表上一个（批量化的）归一化概率分布（即形状为 `(batch_size, sequence_length, vocab_size)` 的 PyTorch 张量），其中预测的分布是对每个输入 token 之后的下一个词的分布。训练语言模型时，我们用这些下一词预测来计算真实的下一个词与预测的下一个词之间的交叉熵损失。在推理阶段从语言模型生成文本时，我们取最后一个时间步（即序列中的最后一项）预测的下一词分布来生成序列中的下一个 token（例如取概率最高的 token、从分布中采样等），把生成的 token 追加到输入序列中，然后重复这一过程。

在作业的这一部分，你将从零开始构建这个 Transformer 语言模型。我们先给出模型的高层次描述，再逐步深入各个组件的细节。

### 3.1 Transformer 语言模型

给定一个 token ID 序列，Transformer 语言模型使用输入嵌入（input embedding）把 token ID 转换为稠密向量，让嵌入后的 token 通过 `num_layers` 个 Transformer 块，然后应用一个可学习的线性投影（“输出嵌入”或“LM head”）产生预测的下一 token logits。示意图见图 1。

#### Token 嵌入

在最开始的一步，Transformer 把（批量化的）token ID 序列嵌入为一个向量序列，这些向量包含 token 身份的信息（图 1 中的红色方块）。

更具体地说，给定一个 token ID 序列，Transformer 语言模型使用 token 嵌入层产生一个向量序列。每个嵌入层接收形状为 `(batch_size, sequence_length)` 的整数张量，产生形状为 `(batch_size, sequence_length, d_model)` 的向量序列。

#### Pre-norm Transformer 块

嵌入之后，激活值会经过若干结构完全相同的神经网络层。一个标准的 decoder-only Transformer 语言模型由 `num_layers` 个相同的层组成（通常称为 Transformer “块”）。每个 Transformer 块接收形状为 `(batch_size, sequence_length, d_model)` 的输入，并返回形状为 `(batch_size,`

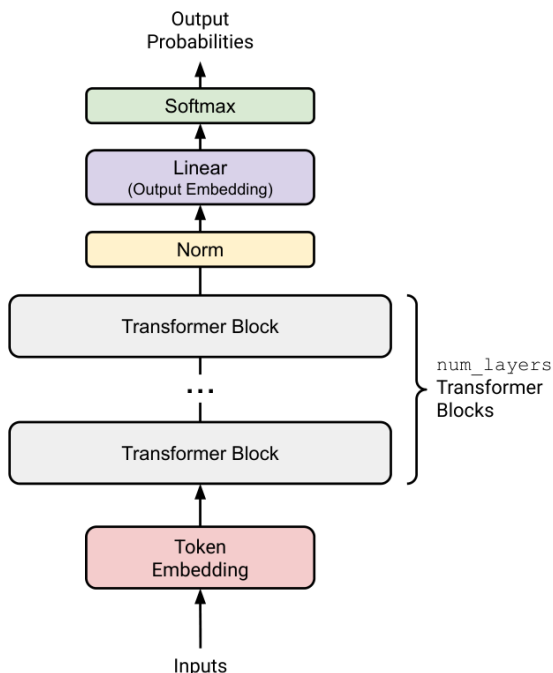


图 1: 我们的 Transformer 语言模型概览。

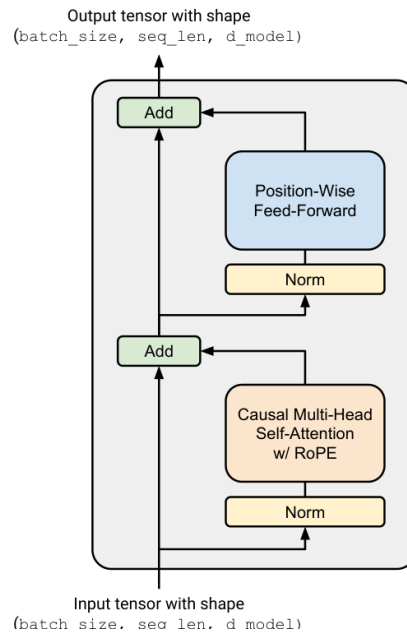


图 2: pre-norm Transformer 块。

`sequence_length`, `d_model`) 的输出。每个块在序列维度上聚合信息（通过自注意力），并对其进行非线性变换（通过前馈层）。

经过 `num_layers` 个 Transformer 块之后，我们把最终的激活值转换为词表上的一个分布。

我们将实现“pre-norm”Transformer 块（详见第 3.4 节），它还要求在最后一个 Transformer 块之后使用层归一化（layer normalization，详见下文），以确保其输出具有合适的尺度。

经过这次归一化之后，我们使用一个标准的可学习线性变换，把 Transformer 块的输出转换为预测的下一 token logits（例如参见 A. Radford et al. [7] 方程 2）。

### 3.2 备注：批处理、Einsum 与高效计算

在 Transformer 中，我们自始至终都在对许多类似批量的输入施加相同的计算。下面举几个例子：

- 批次元素：我们对每个批次元素执行相同的 Transformer 前向运算。
- 序列长度：像 RMSNorm 和前馈网络这类“逐位置”运算对序列的每个位置执行相同的操作。
- 注意力头：注意力运算在“多头”注意力运算中跨注意力头批量执行。

掌握一种符合人体工程学的方式来执行这类运算非常有用，它既要充分利用 GPU，又要易于阅读和理解。许多 PyTorch 运算可以在张量开头接受额外的“批量型”维度，并高效地在这些维度上重复/广播该运算。

例如，假设我们在做一个逐位置的批量运算。我们有一个形状为 `(batch_size, sequence_length, d_model)` 的”数据张量”  $D$ ，想让它与一个形状为 `(d_model, d_model)` 的矩阵  $A$  做批量化的向量-矩阵乘法。此时 `D @ A` 会执行批量矩阵乘法——这是 PyTorch 中的高效原语，其中 `(batch_size, sequence_length)` 维度被批量处理。

因此，假设你的函数可能会收到额外的批量型维度，并把这些维度保持在 PyTorch 形状的开头是很有帮助的。为了让张量能以这种方式批量处理，可能需要通过多步 `view`、`reshape` 和 `transpose` 来整形。这会很麻烦，而且代码在做什么、张量的形状是什么往往会变得难以阅读。

一个更符合人体工程学的选择是在 `torch.einsum` 中使用 `einsum` 记号，或者使用框架无关的库如 `einops` 或 `einx`。两个关键运算是 `einsum`（可以对输入张量的任意维度做张量缩并）和 `rearrange`（可以重排、拼接和拆分任意维度）。事实证明，机器学习中几乎所有运算都是”维度整理 + 张量缩并”的某种组合，偶尔加上（通常是逐点的）非线性函数。这意味着使用 `einsum` 记号可以让你的代码更易读、更灵活。

我们强烈建议在本课程中学习并使用 `einsum` 记号。以前没接触过 `einsum` 记号的同学应当使用 `einops`（[文档在此](#)），已经熟悉 `einops` 的同学应当学习更通用的 `einx`（[在此](#)）。<sup>4</sup> 这两个包都已安装在我们提供的环境中。

下面我们给出一些如何使用 `einsum` 记号的例子。它们是 `einops` 文档的补充，你应当先阅读官方文档。

**示例 (`einstein_example1`): 用 `einops.einsum` 做批量矩阵乘法**

```
import torch
from einops import rearrange, einsum

## 基础实现
Y = D @ A.T
# 很难看出输入和输出的形状及其含义。
# D 和 A 可以是什么形状？其中是否有意外的行为？

## Einsum 自带文档且更稳健
# D A -> Y
Y = einsum(D, A, "batch sequence d_in, d_out d_in -> batch sequence d_out")

## 或者，一个批量化的版本：D 可以有任意前置维度，但 A 受到约束。
Y = einsum(D, A, "... d_in, d_out d_in -> ... d_out")
```

<sup>4</sup>值得注意的是，`einops` 支持非常广泛，而 `einx` 没有经过那么多实战检验。如果你发现 `einx` 有任何限制或 bug，尽管退回使用 `einops` 加一些普通的 PyTorch 代码。

示例 (einstein\_example2): 用 einops.rearrange 做广播运算

我们有一批图像, 想为每张图像基于某个缩放因子生成 10 个变暗版本:

```
images = torch.randn(64, 128, 128, 3) # (batch, height, width, channel)
dim_by = torch.linspace(start=0.0, end=1.0, steps=10)

## 先 reshape 再相乘
dim_value = rearrange(dim_by, "dim_value -> 1 dim_value 1 1 1")
images_rearr = rearrange(images, "b height width channel -> b 1 height width channel")
dimmed_images = images_rearr * dim_value

## 或者一步到位:
dimmed_images = einsum(
    images, dim_by,
    "batch height width channel, dim_value -> batch dim_value height width channel"
)
```

示例 (einstein\_example3): 用 einops.rearrange 做像素混合

假设我们有一批图像, 表示为形状为 (batch, height, width, channel) 的张量, 我们想对图像的所有像素执行一个线性变换, 但该变换应对每个通道独立进行。我们的线性变换表示为形状为 (height \* width, height \* width) 的矩阵  $B$ 。

```
channels_last = torch.randn(64, 32, 32, 3) # (batch, height, width, channel)
B = torch.randn(32*32, 32*32)
```

## 为跨所有像素的混合而重排图像张量

```
channels_last_flat = channels_last.view(
    -1, channels_last.size(1) * channels_last.size(2), channels_last.size(3)
)
channels_first_flat = channels_last_flat.transpose(1, 2)
channels_first_flat_transformed = channels_first_flat @ B.T
channels_last_flat_transformed = channels_first_flat_transformed.transpose(1, 2)
channels_last_transformed = channels_last_flat_transformed.view(*channels_last.shape)
而使用 einops 则是:
height = width = 32
## rearrange 取代了笨拙的 torch view + transpose
channels_first = rearrange(
    channels_last,
```

```

    "batch height width channel -> batch channel (height width)"
)
channels_first_transformed = einsum(
    channels_first, B,
    "batch channel pixel_in, pixel_out pixel_in -> batch channel pixel_out"
)
channels_last_transformed = rearrange(
    channels_first_transformed,
    "batch channel (height width) -> batch height width channel",
    height=height, width=width
)

```

或者，如果你想疯狂一点：用 `einx.dot`（`einx` 中与 `einops.einsum` 等价的运算）一步完成：

```
height = width = 32
```

```

channels_last_transformed = einx.dot(
    "batch row_in col_in channel, (row_out col_out) (row_in col_in)"
    "-> batch row_out col_out channel",
    channels_last, B,
    col_in=width, col_out=width
)

```

第一种实现可以通过在前后添加注释来标明输入和输出形状从而得到改进，但这很笨拙且容易出 bug。使用 `einsum` 记号，文档即实现！

`Einsum` 记号可以处理任意的输入批量维度，还有一个关键优点：自文档化。在使用 `einsum` 记号的代码中，输入输出张量的相关形状要清晰得多。对于其余的张量，你可以考虑使用 `Tensor` 类型标注，例如使用 `jaxtyping` 库（它并非 JAX 专属）。

我们将在作业 2 中更多讨论使用 `einsum` 记号的性能影响，但现在只需知道：它们几乎总是比替代方案更好！

### 3.2.1 数学记号与内存顺序

许多机器学习论文在记号中使用行向量，这样的表示与 NumPy 和 PyTorch 默认采用的行主序（row-major）内存顺序配合良好。使用行向量时，线性变换写作

$$y = xW^{\top}, \quad (1)$$

其中  $W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$  按行主序存储， $x \in \mathbb{R}^{1 \times d_{\text{in}}}$  为行向量。注意，这让我们可以通过增加  $x$  的最外层

维度来批量处理输入，也就是说可以用矩阵输入  $X \in \mathbb{R}^{\text{batch} \times d_{\text{in}}}$  替代向量输入  $x$ 。

在线性代数中更常见的是使用列向量，此时线性变换写作

$$y = Wx, \quad (2)$$

其中  $W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$  按行主序存储， $x \in \mathbb{R}^{d_{\text{in}}}$  为列向量。要在这种设定下批量处理输入， $x$  的批次维度必须放在最后，即需要用矩阵  $\tilde{X} \in \mathbb{R}^{d_{\text{in}} \times \text{batch}}$  替代  $x$ 。

本次作业的数学记号将主要使用列向量，因为数学文献通常遵循这种记号。请记住：如果你想使用朴素的矩阵乘法记号，由于 PyTorch 使用行主序内存顺序，你必须像方程 1 的行向量约定那样以转置的方式应用矩阵。如果你使用 einsum 进行线性代数运算，只要轴标签标注正确，这就完全不成问题。顺便一提，其他语言/线性代数包（如 Matlab、Julia 和 Fortran）都使用列主序内存顺序，即批量维度放在最后，而 Python 及相关包采用了 C 标准的行主序。

### 3.3 基础构件：Linear 与 Embedding 模块

#### 3.3.1 参数初始化

有效地训练神经网络通常需要对模型参数进行仔细的初始化——糟糕的初始化会导致梯度消失或梯度爆炸等不良行为。Pre-norm Transformer 对初始化异常鲁棒，但初始化仍会对训练速度和收敛性产生显著影响。由于本作业已经很长，细节留待作业 3 再讲，这里直接给出一些在大多数情况下都应当行之有效的近似初始化。当前请使用：

- Linear 权重： $\mathcal{N}(\mu = 0, \sigma^2 = \frac{2}{d_{\text{in}} + d_{\text{out}}})$ ，在  $[-3\sigma, 3\sigma]$  处截断。
- Embedding： $\mathcal{N}(\mu = 0, \sigma^2 = 1)$ ，在  $[-3, 3]$  处截断。
- RMSNorm： $\mathbf{1}$ （全 1 向量）。

你应当使用 `torch.nn.init.trunc_normal_` 来初始化截断正态分布的权重。

#### 3.3.2 Linear 模块

Linear 层是 Transformer 和一般神经网络的基础构件。首先，你将实现自己的 `Linear` 类，它继承自 `torch.nn.Module` 并执行线性变换：

$$y = Wx. \quad (3)$$

注意，遵循大多数现代 LLM 的做法，我们不包含偏置项。

问题 (linear): 实现 linear 模块 (1 分)

交付内容: 实现一个继承自 `torch.nn.Module` 的 `Linear` 类, 执行线性变换。你的实现应遵循 PyTorch 内置 `nn.Linear` 模块的接口, 只是没有 `bias` 参数。我们推荐以下接口:

`def __init__(self, in_features, out_features, device=None, dtype=None)` 构造线性变换模块。该函数应接受以下参数:

- `in_features: int` 一输入的最后一维大小
- `out_features: int` 一输出的最后一维大小
- `device: torch.device | None = None` 一参数存放的设备
- `dtype: torch.dtype | None = None` 一参数的数据类型

`def forward(self, x: torch.Tensor) -> torch.Tensor` 对输入应用线性变换。

务必做到:

- 继承 `nn.Module`
- 调用父类构造函数
- 以  $W$  (而非  $W^T$ ) 的形式构造并保存参数, 放入 `nn.Parameter`
- 当然, 不要使用 `nn.Linear` 或 `nn.functional.linear`

初始化时, 使用上文给出的设置, 并配合 `torch.nn.init.trunc_normal_` 初始化权重。

要测试你的 `Linear` 模块, 请实现 `[adapters.run_linear]` 处的测试适配器。适配器应把给定的权重加载到你的 `Linear` 模块中, 为此可以使用 `Module.load_state_dict`。然后运行 `uv run pytest -k test_linear`。

### 3.3.3 Embedding 模块

如上所述, Transformer 的第一层是嵌入层, 它把整数 token ID 映射到维度为 `d_model` 的向量空间。我们将实现一个继承自 `torch.nn.Module` 的自定义 `Embedding` 类 (因此不应使用 `nn.Embedding`)。 `forward` 方法应通过用形状为 `(batch_size, sequence_length)` 的 `torch.LongTensor` token ID 索引形状为 `(vocab_size, d_model)` 的嵌入矩阵, 为每个 token ID 选出对应的嵌入向量。

问题 (embedding): 实现 embedding 模块 (1 分)

交付内容: 实现继承自 `torch.nn.Module` 的 `Embedding` 类, 执行嵌入查找。你的实现应遵循 PyTorch 内置 `nn.Embedding` 模块的接口。我们推荐以下接口:

`def __init__(self, num_embeddings, embedding_dim, device=None, dtype=None)` 构造嵌入模块。该函数应接受以下参数:

- `num_embeddings: int` 一词表大小
- `embedding_dim: int` 一嵌入向量的维度, 即  $d_{\text{model}}$

- `device: torch.device | None = None` — 参数存放的设备
- `dtype: torch.dtype | None = None` — 参数的数据类型

`def forward(self, token_ids: torch.Tensor) -> torch.Tensor` 为给定的 token ID 查找嵌入向量。

务必做到：

- 继承 `nn.Module`
- 调用父类构造函数
- 把嵌入矩阵初始化为 `nn.Parameter`
- 存储嵌入矩阵时让 `d_model` 作为最后一维
- 当然，不要使用 `nn.Embedding` 或 `nn.functional.embedding`

同样，初始化时使用上文给出的设置，并用 `torch.nn.init.trunc_normal_` 初始化权重。

要测试你的实现，请实现 `[adapters.run_embedding]` 处的测试适配器。然后运行 `uv run pytest -k test_embedding`。

### 3.4 Pre-Norm Transformer 块

每个 Transformer 块有两个子层：多头自注意力机制和逐位置前馈网络（A. Vaswani et al. [8], 3.1 节）。

在原初的 Transformer 论文中，模型在两个子层周围各使用一个残差连接，随后是层归一化。这种架构通常称为“post-norm”Transformer，因为层归一化作用于子层的输出。然而，多项研究发现，把层归一化从每个子层的输出移到每个子层的输入（并在最后一个 Transformer 块之后再加一层归一化）可以改善 Transformer 的训练稳定性 [T. Q. Nguyen et al., 2019; R. Xiong et al., 2020]——这一“pre-norm”Transformer 块的可视化表示见图 2。此时，每个 Transformer 块子层的输出通过残差连接与子层输入相加（A. Vaswani et al. [8], 5.4 节）。对 pre-norm 的一个直观理解是：从输入嵌入到 Transformer 最终输出之间存在一条不带任何归一化的干净“残差流”（residual stream），据称这能改善梯度流动。这种 pre-norm Transformer 是当今语言模型使用的标准（例如 GPT-3、LLaMA、PaLM 等），因此我们将实现这一变体。接下来我们将逐一讲解 pre-norm Transformer 块的各个组件，并按顺序实现它们。

#### 3.4.1 均方根层归一化（RMSNorm）

A. Vaswani et al. [8] 的原始 Transformer 实现使用层归一化 [J. L. Ba et al., 2016] 来归一化激活值。遵循 H. Touvron et al. [12]，我们将使用均方根层归一化（root mean square layer normalization, RMSNorm; B. Zhang et al. [13], 方程 4）来做层归一化。给定一个激活值向量  $a \in \mathbb{R}^{d_{\text{model}}}$ , RMSNorm 按下式重新缩放每个激活值  $a_i$ ：

$$\text{RMSNorm}(a)_i = \frac{a_i}{\text{RMS}(a)} g_i, \quad (4)$$

其中  $\text{RMS}(a) = \sqrt{\frac{1}{d_{\text{model}}} \sum_{i=1}^{d_{\text{model}}} a_i^2 + \varepsilon}$ 。这里  $g_i$  是可学习的“增益”(gain) 参数 (总共有  $d_{\text{model}}$  个这样的参数),  $\varepsilon$  是一个超参数, 通常固定为  $1e-5$ 。

你应当把输入向上转型 (upcast) 为 `torch.float32`, 以防止对输入平方时发生溢出。总体而言, 你的 `forward` 方法应当形如:

```
in_dtype = x.dtype
x = x.to(torch.float32)
# 在这里编写执行 RMSNorm 的代码
...
result = ...
# 以原始数据类型返回结果
return result.to(in_dtype)
```

**问题 (rmsnorm):** 均方根层归一化 (1 分)

**交付内容:** 将 `RMSNorm` 实现为一个 `torch.nn.Module`。我们推荐以下接口:

`def __init__(self, d_model: int, eps: float = 1e-5, device=None, dtype=None)` 构造 `RMSNorm` 模块。该函数应接受以下参数:

- `d_model: int` — 模型的隐藏维度
- `eps: float = 1e-5` — 用于数值稳定性的 epsilon 值
- `device: torch.device | None = None` — 参数存放的设备
- `dtype: torch.dtype | None = None` — 参数的数据类型

`def forward(self, x: torch.Tensor) -> torch.Tensor` 处理形状为 `(batch_size, sequence_length, d_model)` 的输入张量, 返回相同形状的张量。

**注意:** 如上所述, 请记住在执行归一化之前把输入向上转型为 `torch.float32` (之后再向下转型回原始数据类型)。

要测试你的实现, 请实现 `[adapters.run_rmsnorm]` 处的测试适配器。然后运行 `uv run pytest -k test_rmsnorm`。

### 3.4.2 逐位置前馈网络

在原初的 Transformer 论文 (A. Vaswani et al. [8] 的 3.3 节) 中, Transformer 前馈网络由两个线性变换组成, 中间夹着一个 ReLU 激活 ( $\text{ReLU}(x) = \max(0, x)$ )。在那个最初的架构中, 前馈内层的维度通常是输入维度的 4 倍。

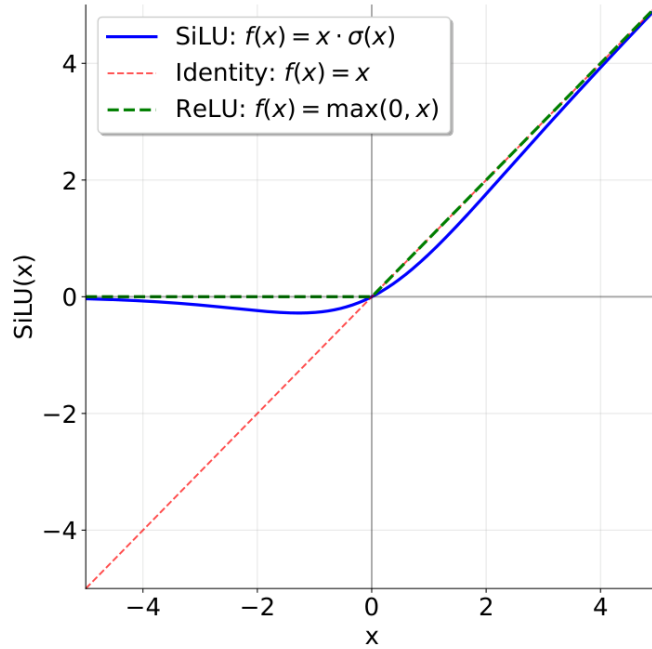


图 3: SiLU（又称 Swish）与 ReLU 激活函数的比较。

然而，与这一原始设计相比，现代语言模型倾向于引入两个主要变化：使用另一种激活函数，以及采用门控机制。具体来说，我们将实现 Llama 3 [A. Grattafiori et al., 2024] 和 Qwen 2.5 [A. Yang et al., 2024] 等 LLM 采用的“SwiGLU”激活函数，它将 SiLU（常称为 Swish）激活与一种称为门控线性单元（Gated Linear Unit, GLU）的门控机制相结合。我们还会省略线性层中有时使用的偏置项，这遵循自 PaLM [A. Chowdhery et al., 2022] 和 LLaMA [H. Touvron et al., 2023] 以来的大多数现代 LLM。

SiLU 或 Swish 激活函数 [D. Hendrycks et al., 2016; S. Elfwing et al., 2017] 定义如下：

$$\text{SiLU}(x) = x \cdot \sigma(x) = \frac{x}{1 + e^{-x}} \quad (5)$$

如图 3 所示，SiLU 激活函数与 ReLU 激活函数类似，但在零点处是平滑的。

门控线性单元（GLU）最初由 Y. N. Dauphin et al. [19] 定义为一个经过 sigmoid 函数的线性变换与另一个线性变换的逐元素乘积：

$$\text{GLU}(x, W_1, W_2) = \sigma(W_1 x) \odot W_2 x, \quad (6)$$

其中  $\odot$  表示逐元素乘法。门控线性单元被认为可以“通过为梯度提供一条线性路径同时保留非线性能力，来缓解深层架构的梯度消失问题”。

把 SiLU/Swish 与 GLU 结合起来，我们就得到 SwiGLU，我们将把它用于前馈网络：

$$\text{FFN}(x) = \text{SwiGLU}(x, W_1, W_2, W_3) = W_2 (\text{SiLU}(W_1 x) \odot W_3 x), \quad (7)$$

其中  $x \in \mathbb{R}^{d_{\text{model}}}$ ,  $W_1, W_3 \in \mathbb{R}^{d_{\text{ff}} \times d_{\text{model}}}$ ,  $W_2 \in \mathbb{R}^{d_{\text{model}} \times d_{\text{ff}}}$ , 并且约定  $d_{\text{ff}} = \frac{8}{3}d_{\text{model}}$ 。对于具体实现，把它取整到附近的 64 的倍数以保证硬件效率是没有问题的。

N. Shazeer [20] 首先提出将 SiLU/Swish 激活与 GLU 结合，并通过实验表明 SwiGLU 在语言建模任务上优于 ReLU 和 SiLU（无门控）等基线。在本作业后面，你将比较 SwiGLU 与 SiLU。虽然我们提到关于这些组件的一些启发式论证（论文也提供了更多支持证据），但保持一种经验性的视角是有益的：Shazeer 论文中如今广为流传的一句话是

“对于这些架构为何似乎有效，我们不提供任何解释；我们把它们的成功，与世间万物一样，归功于上天的仁慈。”（“We offer no explanation as to why these architectures seem to work; we attribute their success, as all else, to divine benevolence.”）

**问题 (positionwise\_feedforward): 实现逐位置前馈网络 (2 分)**

**交付内容:** 实现 SwiGLU 前馈网络，它由一个 SiLU 激活函数和一个 GLU 组成。

**注意:** 在这一特定情况下，为了数值稳定性，你尽可以在实现中使用 `torch.sigmoid`。

你应当在实现中把  $d_{\text{ff}}$  设为约  $\frac{8}{3} \times d_{\text{model}}$ ，同时确保前馈内层的维度是 64 的倍数，以充分利用硬件。要用我们提供的测试来检验你的实现，你需要实现 `[adapters.run_swiglu]` 处的测试适配器。然后运行 `uv run pytest -k test_swiglu` 来测试你的实现。

### 3.4.3 相对位置嵌入

为了向模型注入位置信息，我们将实现旋转位置嵌入（Rotary Position Embeddings）[J. Su et al., 2021]，通常称为 RoPE。对于位于 token 位置  $i$  的给定 query token  $q^{(i)} = W_q x^{(i)} \in \mathbb{R}^d$ ，我们施加一个成对旋转矩阵  $R_i$ ，得到  $q'^{(i)} = R_i q^{(i)} = R_i W_q x^{(i)}$ 。这里  $R_i$  会把嵌入元素对  $q_{2k-1:2k}^{(i)}$  作为二维向量旋转角度  $\theta_{i,k} = i/\Theta^{(2k-2)/d}$ ，其中  $k \in \{1, \dots, d/2\}$ ， $\Theta$  为某个常数。因此，我们可以把  $R_i$  看作一个  $d \times d$  的块对角矩阵，其块为  $R_i^k$  ( $k \in \{1, \dots, \frac{d}{2}\}$ )，其中

$$R_i^k = \begin{pmatrix} \cos(\theta_{i,k}) & -\sin(\theta_{i,k}) \\ \sin(\theta_{i,k}) & \cos(\theta_{i,k}) \end{pmatrix} \quad (8)$$

于是我们得到完整的旋转矩阵

$$R_i = \begin{pmatrix} R_i^1 & 0 & 0 & \cdots & 0 \\ 0 & R_i^2 & 0 & \cdots & 0 \\ 0 & 0 & R_i^3 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & R_i^{(d/2)} \end{pmatrix}, \quad (9)$$

其中 0 表示  $2 \times 2$  的零矩阵。虽然可以构造完整的  $d \times d$  矩阵，但一个好的解法应当利用该矩阵的性质来实现更高效的变换。由于我们只关心给定序列内 token 的相对旋转，我们可以在不同层之间、不同批次之间复用为  $\cos(\theta_{i,k})$  和  $\sin(\theta_{i,k})$  计算的值。如果你想优化它，可以使用一个由所有层共同引用的单一 RoPE 模块，它可以在 init 期间通过 `self.register_buffer(persistent=False)` 创建一个 sin 和 cos 值的二维预计算缓冲区 (buffer)，而不是 `nn.Parameter` (因为我们不想学习这些固定的余弦和正弦值)。接下来，我们对  $q^{(i)}$  所做的完全相同的旋转过程也施加于  $k^{(j)}$ ，按相应的  $R_j$  旋转。注意，这一层没有可学习的参数。

#### 问题 (rope): 实现 RoPE (2 分)

**交付内容:** 实现一个 `RotaryPositionalEmbedding` 类，对输入张量施加 RoPE。

推荐以下接口:

`def __init__(self, theta: float, d_k: int, max_seq_len: int, device=None)` 构造 RoPE 模块，并在需要时创建缓冲区。

- `theta: float` — RoPE 的  $\Theta$  值
- `d_k: int` — query 和 key 向量的维度
- `max_seq_len: int` — 允许输入的最大序列长度
- `device: torch.device | None = None` — 缓冲区存放的设备

`def forward(self, x: torch.Tensor, token_positions: torch.Tensor) ->`

`torch.Tensor` 处理形状为 `(..., seq_len, d_k)` 的输入张量，返回相同形状的张量。注意，你应当容忍带有任意数量批量维度的  $x$ 。你应当假设 `token` 位置是形状为 `(..., seq_len)` 的张量，指定  $x$  沿序列维度的 token 位置。

你应当使用 `token` 位置沿序列维度对你的 (可能预计算的) `cos` 和 `sin` 张量进行切片。

要测试你的实现，请完成 `[adapters.run_rope]` 并确保它通过 `uv run pytest -k test_rope`。

#### 3.4.4 缩放点积注意力

现在我们将实现 A. Vaswani et al. [8] (3.2.1 节) 所述的缩放点积注意力。作为预备步骤，Attention 运算的定义会用到 softmax——一种把未归一化的分数向量转换为归一化分布的运算:

$$\text{softmax}(v)_i = \frac{\exp(v_i)}{\sum_{j=1}^n \exp(v_j)}. \quad (10)$$

注意，当数值很大时  $\exp(v_i)$  可能变为  $\text{inf}$ （此时  $\frac{\infty}{\infty} = \text{NaN}$ ）。我们可以注意到  $\text{softmax}$  运算对给所有输入加上任意常数  $c$  保持不变，从而避免这个问题。我们可以利用这一性质实现数值稳定性——通常，我们会从  $v$  的所有元素中减去  $v$  的最大元素，使新的最大元素为 0。现在你将使用这个数值稳定性技巧实现  $\text{softmax}$ 。

**问题 (softmax): 实现 softmax (1 分)**

**交付内容:** 编写一个对张量应用  $\text{softmax}$  运算的函数。你的函数应接收两个参数：一个张量和一个维度  $i$ ，并对输入张量的第  $i$  个维度应用  $\text{softmax}$ 。输出张量应与输入张量形状相同，但其第  $i$  个维度现在是一个归一化的概率分布。请使用“从第  $i$  个维度的所有元素中减去该维度最大值”的技巧来避免数值稳定性问题。

要测试你的实现，请完成 `[adapters.run_softmax]` 并确保它通过 `uv run pytest -k test_softmax_matches_pytorch`。

现在我们可以从数学上定义 Attention 运算：

$$\text{Attention}(Q, K, V) = \text{softmax} \left( \frac{QK^\top}{\sqrt{d_k}} \right) V \quad (11)$$

其中  $Q \in \mathbb{R}^{n \times d_k}$ ,  $K \in \mathbb{R}^{m \times d_k}$ ,  $V \in \mathbb{R}^{m \times d_v}$ 。这里  $Q$ 、 $K$ 、 $V$  都是该运算的输入——注意它们不是可学习的参数。

**掩码 (Masking)** : 有时遮蔽 (mask) 注意力运算的输出会带来便利。掩码的形状应为  $M \in \{\text{True}, \text{False}\}^{n \times m}$ ，这个布尔矩阵的第  $i$  行指示 query  $i$  应当关注哪些 key。按惯例（可能有点令人困惑），位置  $(i, j)$  上的值  $\text{True}$  表示 query  $i$  确实关注 key  $j$ ，值  $\text{False}$  表示 query 不关注该 key。换言之，“信息流动”发生在值为  $\text{True}$  的  $(i, j)$  位置上。例如，考虑一个  $1 \times 3$  的掩码矩阵，其条目为 `[[True, True, False]]`：这个单独的 query 向量只关注前两个 key。

从计算角度看，使用掩码比对子序列计算注意力高效得多；做法是取  $\text{pre-softmax}$  的值  $(QK^\top / \sqrt{d_k})$  并把  $-\infty$  加到掩码矩阵中为  $\text{False}$  的任何条目上。

**问题 (scaled\_dot\_product\_attention): 实现缩放点积注意力 (5 分)**

**交付内容:** 实现缩放点积注意力函数。你的实现应能处理形状为 `(batch_size, ..., seq_len, d_k)` 的 keys 和 queries，以及形状为 `(batch_size, ..., seq_len, d_v)` 的 values，其中 `...` 表示任意数量的其他批量型维度（如果提供）。实现应返回形状为 `(batch_size, ..., seq_len, d_v)` 的输出。关于批量型维度的讨论见第 3.2 节。

你的实现还应支持一个可选的、用户提供的形状为  $(\text{seq\_len}, \text{seq\_len})$  的布尔掩码。掩码值为 True 的位置的注意力概率合计应为 1，掩码值为 False 的位置的注意力概率应为 0。

要用我们提供的测试来检验你的实现,你需要实现 `[adapters.run_scaled_dot_product_attention]` 处的测试适配器。`uv run pytest -k test_scaled_dot_product_attention` 会在三阶输入张量上测试你的实现, 而 `uv run pytest -k test_4d_scaled_dot_product_attention` 会在四阶输入张量上测试你的实现。

### 3.4.5 因果多头自注意力

我们将实现 A. Vaswani et al. [8] 的 3.2.2 节所述的多头自注意力。回顾一下, 多头注意力运算在数学上定义如下:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) \quad (12)$$

其中

$$\text{head}_i = \text{Attention}(Q_i, K_i, V_i) \quad (13)$$

其中  $Q_i, K_i, V_i$  分别是  $Q, K, V$  的嵌入维度上大小为  $d_k$  或  $d_v$  的第  $i \in \{1, \dots, h\}$  个切片,  $\text{Attention}$  为第 3.4.4 节定义的缩放点积注意力运算。由此我们可以构成多头自注意力运算:

$$\text{MultiHeadSelfAttention}(x) = W_O \text{MultiHead}(W_Q x, W_K x, W_V x) \quad (14)$$

这里可学习的参数是  $W_Q \in \mathbb{R}^{hd_k \times d_{\text{model}}}$ 、 $W_K \in \mathbb{R}^{hd_k \times d_{\text{model}}}$ 、 $W_V \in \mathbb{R}^{hd_v \times d_{\text{model}}}$  和  $W_O \in \mathbb{R}^{d_{\text{model}} \times hd_v}$ 。由于  $Q, K, V$  在多头注意力运算中被切片, 我们可以认为  $W_Q, W_K, W_V$  沿输出维度按每个头分开。当你把它调通后, 你应当是用总共三次矩阵乘法来计算 key、value 和 query 投影。<sup>5</sup>

### 因果掩码 (Causal masking)

你的实现应当阻止模型关注序列中未来的 token。换言之, 如果给模型一个 token 序列  $t_1, \dots, t_n$ , 而我们要为前缀  $t_1, \dots, t_i$  (其中  $i < n$ ) 计算下一词预测, 那么模型不应能访问 (关注) 位置  $t_{i+1}, \dots, t_n$  上的 token 表示, 因为推理生成文本时它无法访问这些 token (而且这些未来 token 会泄露真实下一个词身份的信息, 使语言建模预训练目标变得毫无意义)。对于输入 token 序列  $t_1, \dots, t_n$ , 朴素的阻止访问未来 token 的方法是运行  $n$  次多头自注意力 (对序列中的  $n$  个不同前缀各一次)。相反, 我们将使用因果注意力掩码 (causal attention masking), 它允许 token  $i$  关注序列中所有位置  $j \leq i$ 。你

<sup>5</sup>作为进阶目标, 试着把 key、query、value 投影合并成一个权重矩阵, 这样你只需要一次矩阵乘法。

可以使用 `torch.triu` 或广播化的索引比较来构造这个掩码，并且你应当利用这样一个事实：你在第 3.4.4 节实现的缩放点积注意力已经支持注意力掩码。

### 应用 RoPE

RoPE 应施加于 query 和 key 向量，而不施加于 value 向量。此外，头维度应作为批量维度处理，因为在多头注意力中，注意力是对每个头独立施加的。这意味着对每个头的 query 和 key 向量施加的是完全相同的 RoPE 旋转。

**问题 (multihead\_self\_attention): 实现因果多头自注意力 (5 分)**

**交付内容:** 将因果多头自注意力实现为一个 `torch.nn.Module`。你的实现应当（至少）接受以下参数：

- `d_model: int` —Transformer 块输入的维度。
- `num_heads: int` —多头自注意力使用的头数。

遵循 A. Vaswani et al. [8]，设  $d_k = d_v = \frac{d_{\text{model}}}{h}$ 。要用我们提供的测试来检验你的实现，请实现 `[adapters.run_multihead_self_attention]` 处的测试适配器。然后运行 `uv run pytest -k test_multihead_self_attention` 来测试你的实现。

## 3.5 完整的 Transformer 语言模型

让我们从组装 Transformer 块开始（参阅图 2 会很有帮助）。一个 Transformer 块包含两个”子层”：一个用于多头自注意力，另一个用于 SwiGLU 前馈网络。在每个子层中，我们先执行 RMSNorm，然后执行主运算（MHA/FF），最后加入残差连接。

具体而言，Transformer 块的前半部分（第一个”子层”）应当实现如下一组更新，从输入  $x$  产生输出  $y$ ：

$$y = x + \text{MultiHeadSelfAttention}(\text{RMSNorm}(x)). \quad (15)$$

**问题 (transformer\_block): 实现 Transformer 块 (3 分)**

按照第 3.4 节的描述和图 2 的图示实现 pre-norm Transformer 块。你的 Transformer 块应当（至少）接受以下参数：

- `d_model: int` —Transformer 块输入的维度。
- `num_heads: int` —多头自注意力使用的头数。
- `d_ff: int` —逐位置前馈网络内层的维度。

要测试你的实现，请实现适配器 `[adapters.run_transformer_block]`。然后运行 `uv run`

`pytest -k test_transformer_block` 来测试你的实现。

**交付内容：**能通过所提供测试的 Transformer 块代码。

现在按照图 1 的高层示意图把各个块组装起来。遵循第 3.1.0.1 节中对嵌入的描述，将其送入 `num_layers` 个 Transformer 块，然后传入最后的层归一化和 LM head，得到词表上的未归一化分布（即 logits）。

**问题 (transformer\_lm)：实现 Transformer 语言模型 (3 分)**

是时候把它们全部拼在一起了！按照第 3.1 节的描述和图 1 的图示实现 Transformer 语言模型。至少，你的实现应当接受 Transformer 块的所有前述构造参数，以及这些附加参数：

- `vocab_size: int` — 词表大小，用于确定 token 嵌入矩阵的维度。
- `context_length: int` — 最大上下文长度，用于确定 RoPE 的 sin 和 cos 缓冲区的维度。
- `num_layers: int` — 使用的 Transformer 块数量。

要用我们提供的测试来检验你的实现，你需要先实现 `[adapters.run_transformer_lm]` 处的测试适配器。然后运行 `uv run pytest -k test_transformer_lm` 来测试你的实现。

**交付内容：**能通过上述测试的 Transformer 语言模型模块。

#### 资源核算

能够理解 Transformer 各部分如何消耗算力和内存是很有用的。我们将走一遍进行基本“FLOPs 核算”的步骤。Transformer 中绝大多数 FLOPS 来自矩阵乘法，所以我们的核心方法很简单：

1. 写下 Transformer 前向传播中的所有矩阵乘法。
2. 把每个矩阵乘法换算成所需的 FLOPs。

第二步中，下面的事实会很有用：

**法则：**给定  $A \in \mathbb{R}^{m \times n}$  和  $B \in \mathbb{R}^{n \times p}$ ，矩阵乘积  $AB$  需要  $2mnp$  FLOPs。

要理解这一点，注意  $(AB)[i, j] = A[i, :] \cdot B[:, j]$ ，而这个点积需要  $n$  次加法和  $n$  次乘法（ $2n$  FLOPs）。又因为矩阵乘积  $AB$  有  $m \times p$  个条目，所以 FLOPS 总数为  $(2n)(mp) = 2mnp$ 。

现在，在做下一道题之前，逐一过一遍你的 Transformer 块和 Transformer 语言模型的每个组件，列出所有矩阵乘法及其相应的 FLOPs 开销，会很有帮助。

**问题 (transformer\_accounting)：Transformer 语言模型资源核算 (5 分)**

(a) 考虑一个采用我们作业架构的 GPT-2 XL 规模的模型，它具有如下配置：

- `vocab_size: 50,257`
- `context_length: 1,024`
- `num_layers: 48`
- `d_model: 1,600`
- `num_heads: 25`

- $d_{ff}$ : 4,288 (最接近  $\frac{8}{3} \times 1,600$  的 64 的倍数)

假设我们用该配置构建模型。我们的模型会有多少可训练参数？假设每个参数用单精度浮点数表示，仅加载该模型需要多少内存？

**交付内容：**一到两句话的回答。

- (b) 找出完成我们的 GPT-2 XL 形状模型一次前向传播所需的矩阵乘法。这些矩阵乘法总共需要多少 FLOPs？假设我们的输入序列有 `context_length` 个 token。

**交付内容：**一个矩阵乘法列表（附说明）以及所需 FLOPs 的总数。

- (c) 根据你上面的分析，模型的哪些部分需要最多的 FLOPs？

**交付内容：**一到两句话的回答。

- (d) 用 GPT-2 small (12 层, 768 `d_model`, 12 头)、GPT-2 medium (24 层, 1024 `d_model`, 16 头) 和 GPT-2 large (36 层, 1280 `d_model`, 20 头) 重复你的分析。随着模型规模增大，Transformer 语言模型的哪些部分占总 FLOPs 的比例会变大或变小？

**交付内容：**对每个模型，给出模型组件的分解及其对应的 FLOPs (占一次前向传播所需总 FLOPs 的比例)。另外，用一到两句话描述模型规模的变化如何改变各组件的比例 FLOPs。

- (e) 取 GPT-2 XL 并把上下文长度增加到 16,384。一次前向传播的总 FLOPs 如何变化？各模型组件的 FLOPs 相对占比如何变化？

**交付内容：**一到两句话的回答。

## 4 训练 Transformer 语言模型

现在我们有了预处理数据（通过分词器）和模型（Transformer）的方法。剩下的就是构建支持训练的全部代码。它由以下部分组成：

- 损失：我们需要定义损失函数（交叉熵）。
- 优化器：我们需要定义最小化该损失的优化器（AdamW）。
- 训练循环：我们需要所有配套的基础设施，用于加载数据、保存检查点以及管理训练。

### 4.1 交叉熵损失

回顾一下，Transformer 语言模型为每条长度为  $m+1$  的序列  $x$  和每个  $i = 1, \dots, m$  定义分布  $p_\theta(x_{i+1} | x_{1:i})$ 。给定一个由长度为  $m+1$  的序列构成的训练集  $D$ ，我们定义标准的交叉熵（负对数似然）损失函数：

$$\ell(\theta; D) = \frac{1}{|D|} \sum_{x \in D} \sum_{i=1}^m -\log p_{\theta}(x_{i+1} | x_{1:i}). \quad (16)$$

(注意, Transformer 的一次前向传播就能得到所有  $i = 1, \dots, m$  的  $p_{\theta}(x_{i+1} | x_{1:i})$ 。)

特别地, Transformer 为每个位置  $i$  计算 logits  $o_i \in \mathbb{R}^{\text{vocab\_size}}$ , 由此得到:<sup>6</sup>

$$p(x_{i+1} | x_{1:i}) = \text{softmax}(o_i)[x_{i+1}] = \frac{\exp(o_i[x_{i+1}])}{\sum_{a=1}^{\text{vocab\_size}} \exp(o_i[a])}. \quad (17)$$

交叉熵损失一般相对于 logits 向量  $o_i \in \mathbb{R}^{\text{vocab\_size}}$  和目标  $x_{i+1}$  来定义:<sup>7</sup>

实现交叉熵损失时需要像 softmax 那样小心处理数值问题。

**问题 (cross\_entropy): 实现交叉熵 (1 分)**

**交付内容:** 编写一个计算交叉熵损失的函数, 接收预测 logits ( $o_i$ ) 和目标 ( $x_{i+1}$ ), 并计算交叉熵  $\ell_i = -\log \text{softmax}(o_i)[x_{i+1}]$ 。你的函数应能处理:

- 为数值稳定性减去最大元素。
- 尽可能约掉 log 和 exp。
- 处理任意额外的批量维度并返回批次上的平均值。与第 3.2 节一样, 我们假定批量型维度总是位于最前面、在词表大小维度之前。

实现 `[adapters.run_cross_entropy]`, 然后运行 `uv run pytest -k test_cross_entropy` 来测试你的实现。

## 困惑度 (Perplexity)

交叉熵足以用于训练, 但在评估模型时, 我们还想报告困惑度 (perplexity)。对于一条长度为  $m$ 、各位置交叉熵损失为  $\ell_1, \dots, \ell_m$  的序列:

$$\text{perplexity} = \exp \left( \frac{1}{m} \sum_{i=1}^m \ell_i \right). \quad (18)$$

## 4.2 SGD 优化器

有了损失函数之后, 我们开始探索优化器。最简单的基于梯度的优化器是随机梯度下降 (Stochastic Gradient Descent, SGD)。我们从随机初始化的参数  $\theta_0$  出发, 然后对每一步  $t = 0, \dots, T-1$  执行如

<sup>6</sup>注意,  $o_i[k]$  指向量  $o_i$  中索引  $k$  处的值。

<sup>7</sup>这对应于  $x_{i+1}$  上的 Dirac delta 分布与预测分布  $\text{softmax}(o_i)$  之间的交叉熵。

下更新：

$$\theta_{t+1} \leftarrow \theta_t - \alpha_t \nabla L(\theta_t; B_t), \quad (19)$$

其中  $B_t$  是从数据集  $D$  中随机采样的一批数据，学习率  $\alpha_t$  和批量大小  $|B_t|$  是超参数。

#### 4.2.1 在 PyTorch 中实现 SGD

要实现我们的优化器，我们将继承 PyTorch 的 `torch.optim.Optimizer` 类。一个 `Optimizer` 子类必须实现两个方法：

`def __init__(self, params, ...)` 应当初始化你的优化器。这里 `params` 是要优化的参数集合（或参数组，当用户想对模型的不同部分使用不同超参数（例如学习率）时）。务必把 `params` 传给基类的 `__init__` 方法，基类会保存这些参数供 `step` 使用。你可以根据优化器的需要接收额外的参数（例如学习率就是一个常见参数），并把它们作为字典传给基类构造函数，字典的键是你为这些参数选择的名称（字符串）。

`def step(self)` 应当对参数执行一次更新。在训练循环中，它会在反向传播之后被调用，因此你可以访问上一个批次的梯度。该方法应遍历每个参数张量 `p` 并就地修改它们，即根据梯度 `p.grad`（如果存在）设置 `p.data`——`p.data` 保存与该参数关联的张量，`p.grad` 是损失关于该参数的梯度张量。

PyTorch 优化器 API 有一些微妙之处，用一个例子来解释更容易。为了让例子更丰富，我们将实现 SGD 的一个变体：学习率随训练衰减，从初始学习率  $\alpha$  开始，随时间迈出越来越小的步子：

$$\theta_{t+1} = \theta_t - \frac{\alpha}{\sqrt{t+1}} \nabla L(\theta_t; B_t) \quad (20)$$

我们来看看这个版本的 SGD 会如何实现为一个 PyTorch `Optimizer`：

```
from collections.abc import Callable, Iterable
from typing import Optional
import torch
import math

class SGD(torch.optim.Optimizer):
    def __init__(self, params, lr=1e-3):
        if lr < 0:
            raise ValueError(f"Invalid learning rate: {lr}")
```

```

    defaults = {"lr": lr}
    super().__init__(params, defaults)

    def step(self, closure: Optional[Callable] = None):
        loss = None if closure is None else closure()
        for group in self.param_groups:
            lr = group["lr"] # 获取学习率。
            for p in group["params"]:
                if p.grad is None:
                    continue
                state = self.state[p] # 获取与 p 关联的状态。
                t = state.get("t", 0) # 从状态中获取迭代次数，若无则为 0。
                grad = p.grad.data # 获取损失关于 p 的梯度。
                p.data -= lr / math.sqrt(t + 1) * grad # 就地更新权重张量。
                state["t"] = t + 1 # 迭代次数加一。
        return loss

```

在 `__init__` 中，我们把参数以及默认超参数传给基类构造函数（参数可能以组的形式给出，每组带有不同的超参数）。如果参数只是单个 `torch.nn.Parameter` 对象集合，基类构造函数会创建一个组并为其分配默认超参数。然后，在 `step` 中，我们遍历每个参数组、再遍历该组中的每个参数，并应用方程 20。这里，我们把迭代次数保存为与每个参数关联的状态：先读取该值，在梯度更新中使用它，然后再更新它。

API 规定用户可以传入一个可调用的 `closure`，在优化器 `step` 之前重新计算损失。我们要用的优化器并不需要它，但为了遵循 API 我们加上了。

为了看它如何工作，可以使用下面这个最小化的训练循环示例：

```

weights = torch.nn.Parameter(5 * torch.randn((10, 10)))
opt = SGD([weights], lr=1)

for t in range(100):
    opt.zero_grad() # 重置所有可学习参数的梯度。
    loss = (weights**2).mean() # 计算标量损失值。
    print(loss.cpu().item())
    loss.backward() # 运行反向传播，计算梯度。
    opt.step() # 运行优化器一步。

```

这就是典型的训练循环结构：每次迭代中，我们计算损失并执行一步优化器。训练语言模型时，可学习参数来自模型（在 PyTorch 中，`m.parameters()` 给出该集合）。损失会在采样的一批数据上计算，

但训练循环的基本结构是一样的。

**问题 (learning\_rate\_tuning): 调整学习率 (1 分)**

正如我们将看到的，对训练影响最大的超参数之一就是学习率。让我们上面的玩具示例中实际看看。用另外三个学习率  $1e1$ 、 $1e2$  和  $1e3$  运行上面的 SGD 示例，只训练 10 次迭代。每个学习率下损失会发生什么？衰减得更快、更慢，还是发散（即在训练过程中不降反升）？

**交付内容：**一到两句话的回答，说明你观察到的行为。

### 4.3 AdamW

现代语言模型通常使用比 SGD 更复杂的优化器来训练。近来使用的优化器大多是 Adam 优化器 [D. P. Kingma et al., 2015] 的衍生品。我们将使用 AdamW [I. Loshchilov et al., 2019]，它在近期工作中被广泛使用。AdamW 对 Adam 提出了一种修改：以与梯度更新解耦的方式加入权重衰减（在每次迭代中把参数向 0 拉近），从而改善正则化。我们将按照 I. Loshchilov et al. [23] 的算法 2 来实现 AdamW。

AdamW 是有状态的：对每个参数，它维护其一阶矩和二阶矩的运行估计。因此，AdamW 用额外的内存换取更好的稳定性和收敛性。除了学习率  $\alpha$  之外，AdamW 还有一对控制矩估计更新的超参数  $(\beta_1, \beta_2)$ ，以及权重衰减率  $\lambda$ 。典型应用把  $(\beta_1, \beta_2)$  设为  $(0.9, 0.999)$ ，但 LLaMA [H. Touvron et al., 2023] 和 GPT-3 [T. B. Brown et al., 2020] 等大语言模型常用  $(0.9, 0.95)$  训练。该算法可以写作如下，其中  $\varepsilon$  是一个很小的值（例如  $10^{-8}$ ），用于在  $v$  中出现极小值时改善数值稳定性：

**算法 1: AdamW 优化器**

1.  $\text{init}(\theta)$  ——▷ 初始化可学习参数
2.  $m \leftarrow 0$  ——▷ 一阶矩向量的初值；与  $\theta$  形状相同
3.  $v \leftarrow 0$  ——▷ 二阶矩向量的初值；与  $\theta$  形状相同
4. **for**  $t = 1, \dots, T$  **do**
- 5.
- 采样一批数据  $B_t$
- 6.
- $g \leftarrow \nabla_{\theta} \ell(\theta; B_t)$  ——▷ 计算损失的梯度
- 7.
- $\alpha_t \leftarrow \alpha \frac{\sqrt{1-\beta_2^t}}{1-\beta_1^t}$  ——▷ 为迭代  $t$  计算调整后的  $\alpha$
- 8.
- $\theta \leftarrow \theta - \alpha_t \lambda \theta$  ——▷ 应用权重衰减
- 9.
- $m \leftarrow \beta_1 m + (1 - \beta_1)g$  ——▷ 更新一阶矩估计
- 10.
- $v \leftarrow \beta_2 v + (1 - \beta_2)g^2$  ——▷ 更新二阶矩估计
- 11.
- $\theta \leftarrow \theta - \alpha_t \frac{m}{\sqrt{v+\epsilon}}$  ——▷ 应用矩调整后的权重更新
12. **end for**

注意  $t$  从 1 开始。现在你来实现这个优化器。

**问题 (adamw): 实现 AdamW (2 分)**

**交付内容:** 将 AdamW 优化器实现为 `torch.optim.Optimizer` 的子类。你的类应在 `__init__` 中接收学习率  $\alpha$ , 以及  $\beta$ 、 $\epsilon$  和  $\lambda$  超参数。为了帮你保存状态, 基类 `Optimizer` 给了你一个字典 `self.state`, 它把 `nn.Parameter` 对象映射到一个字典, 你可以在其中存储该参数所需的任何信息 (对 AdamW 而言就是矩估计)。实现 `[adapters.get_adamw_cls]` 并确保它通过 `uv run pytest -k test_adamw`。

**问题 (adamw\_accounting): 使用 AdamW 训练的资源核算 (2 分)**

我们来计算运行 AdamW 需要多少内存和算力。假设每个张量都使用 `float32`。

- (a) 运行 AdamW 需要多少峰值内存? 请按参数、激活值、梯度和优化器状态的内存占用分解你的答案。用 `batch_size` 和模型超参数 (`vocab_size`、`context_length`、`num_layers`、

`d_model`、`num_heads`) 来表达你的答案。假设  $d_{\text{ff}} = \frac{8}{3} \times d_{\text{model}}$ 。

为简单起见，在计算激活值的内存占用时，只考虑以下组件：

- Transformer 块
  - RMSNorm(s)
  - 多头自注意力子层： $QKV$  投影、 $QK^T$  矩阵乘法、softmax、values 的加权和、输出投影。
  - 逐位置前馈网络 (SwiGLU)： $W_1$ 、 $W_2$ 、门控分支上的 SiLU、逐元素乘积、 $W_3$
- 最后的 RMSNorm
- 输出嵌入
- logits 上的交叉熵

**交付内容：**参数、激活值、梯度和优化器状态各自的代数表达式，以及总数。

- (b) 把你的答案代入 GPT-2 XL 形状模型，得到一个只依赖于 `batch_size` 的表达式。在 80GB 显存内能容纳的最大批量大小是多少？

**交付内容：**一个形如  $a \cdot \text{batch\_size} + b$  的表达式（其中  $a$ 、 $b$  为数值），以及代表最大批量大小的数字。

- (c) 运行一步 AdamW 需要多少 FLOPs？

**交付内容：**一个代数表达式，并附简要说明。

- (d) 模型 FLOPs 利用率 (model FLOPs utilization, MFU) 定义为观测吞吐量 (每秒 token 数) 与硬件理论峰值 FLOP 吞吐量之比 [A. Chowdhery et al., 2022]。一块 NVIDIA H100 GPU 对 “float32” (实际上是 TensorFloat-32, 现实中是 “bfloat16” 运算) 的理论峰值为 495 teraFLOP/s。假设你能达到 50% 的 MFU, 在单块 H100 上以批量大小 1024 训练一个 GPT-2 XL 达 400K 步需要多长时间？遵循 J. Kaplan et al. [25] 和 J. Hoffmann et al. [26], 假设反向传播的 FLOPs 是前向传播的两倍。

**交付内容：**训练所需的小时数，并附简要说明。

## 4.4 学习率调度

使损失下降最快的学习率取值在训练过程中往往不断变化。训练 Transformer 时，通常使用学习率调度 (learning rate schedule)：开始时使用较大的学习率，在早期进行更快的更新，随着模型训练慢慢衰减到较小的值。<sup>8</sup> 在本作业中，我们将实现用于训练 LLaMA [H. Touvron et al., 2023] 的余弦退火 (cosine annealing) 调度。

调度器就是一个函数：输入当前步  $t$  和其他相关参数（如初始和最终学习率），返回在第  $t$  步用于梯

<sup>8</sup>有时也常见使用让学习率重新回升（重启/restart）的调度，以帮助跳出局部极小值。

度更新的学习率。最简单的调度是常数函数，对任何  $t$  都返回相同的学习率。

余弦退火学习率调度接收：(i) 当前迭代  $t$ ；(ii) 最大学习率  $\alpha_{\max}$ ；(iii) 最小（最终）学习率  $\alpha_{\min}$ ；(iv) 预热（warm-up）迭代次数  $T_w$ ；(v) 余弦退火的最终迭代  $T_c$ 。迭代  $t$  时的学习率定义为：

（预热）若  $t < T_w$ ，则  $\alpha_t = \frac{t}{T_w} \alpha_{\max}$ 。

（余弦退火）若  $T_w \leq t \leq T_c$ ，则  $\alpha_t = \alpha_{\min} + \frac{1}{2} \left( 1 + \cos \left( \frac{t - T_w}{T_c - T_w} \pi \right) \right) (\alpha_{\max} - \alpha_{\min})$ 。

（退火后）若  $t > T_c$ ，则  $\alpha_t = \alpha_{\min}$ 。

**问题 (learning\_rate\_schedule): 实现带预热的余弦学习率调度 (1 分)**

编写一个函数，接收  $t$ 、 $\alpha_{\max}$ 、 $\alpha_{\min}$ 、 $T_w$  和  $T_c$ ，按上面定义的调度器返回学习率  $\alpha_t$ 。然后实现 `[adapters.get_lr_cosine_schedule]` 并确保它通过 `uv run pytest -k test_get_lr_cosine_schedule`。

## 4.5 梯度裁剪

训练时，我们有时会遇到产生大梯度的训练样本，这会使训练不稳定。为缓解这一问题，实践中常用的一种技术是梯度裁剪（gradient clipping）。其思想是在每次反向传播之后、执行优化器 step 之前，对梯度的范数施加一个限制。给定（所有参数的）梯度  $g$ ，我们计算其  $\ell_2$ -范数  $\|g\|_2$ 。若该范数小于最大值  $M$ ，则保持  $g$  不变；否则，我们把  $g$  按因子  $\frac{M}{\|g\|_2 + \varepsilon}$  缩小（其中加上一个小的  $\varepsilon$ （如  $10^{-6}$ ）以保证数值稳定）。注意，裁剪后的范数会略小于  $M$ 。

**问题 (gradient\_clipping): 实现梯度裁剪 (1 分)**

编写一个实现梯度裁剪的函数。你的函数应接收一个参数列表和一个最大  $\ell_2$ -范数，并就地修改每个参数的梯度。使用  $\varepsilon = 10^{-6}$ （PyTorch 默认值）。然后实现适配器 `[adapters.run_gradient_clipping]` 并确保它通过 `uv run pytest -k test_gradient_clipping`。

## 5 训练循环

现在，我们终于可以把目前为止构建的主要组件拼在一起了：分词后的数据、模型和优化器。

## 5.1 数据加载器

分词后的数据（例如你在 `tokenizer_experiments` 中准备的数据）是单个 token 序列  $x = (x_1, \dots, x_n)$ 。即使源数据由多篇独立文档组成（例如不同的网页或源代码文件），常见的做法也是把它们全部拼接为单个 token 序列，并在它们之间加分隔符（例如 `<|endoftext|>` token）。

数据加载器把它变成一个批次流，每个批次由  $B$  条长度为  $m$  的序列以及对应的下一 token（长度同样为  $m$ ）组成。例如，当  $B = 1$ 、 $m = 3$  时， $([x_2, x_3, x_4], [x_3, x_4, x_5])$  就是一个可能的批次。

以这种方式加载数据能从多方面简化训练。首先，任何  $1 \leq i \leq n - m$  都给出一条有效的训练序列，因此采样训练序列轻而易举。由于所有训练序列长度相同，无需对输入序列进行填充（padding），这提高了硬件利用率（也因为可以增大批量大小  $B$ ）。最后，我们也不需要加载整个数据集来采样训练数据，这使得处理那些本来可能装不进内存的大数据集变得容易。

### 问题（data\_loading）：实现数据加载（2 分）

**交付内容：**编写一个函数，接收一个 numpy 数组  $x$ （token ID 的整数数组）、一个 `batch_size`、一个 `context_length` 和一个 PyTorch 设备字符串（例如 `'cpu'` 或 `'cuda:0'`），返回一对张量：采样出的输入序列和对应的下一 token 目标。两个张量的形状都应为  $(\text{batch\_size}, \text{context\_length})$ 、内容为 token ID，且都应放在所请求的设备上。要用我们提供的测试来检验你的实现，你需要先实现 `[adapters.run_get_batch]` 处的测试适配器。然后运行 `uv run pytest -k test_get_batch` 来测试你的实现。

### 低资源建议：在 CPU 或 Apple Silicon 上加载数据

如果你打算在 CPU 或 Apple Silicon 上训练语言模型，你需要把数据移动到正确的设备上（同样，之后你的模型也应使用相同的设备）。

如果你在 CPU 上，可以使用 `'cpu'` 设备字符串；在 Apple Silicon（M\* 芯片）上，可以使用 `'mps'` 设备字符串。

关于 MPS 的更多信息，请参阅以下资源：

- <https://docs.pytorch.org/docs/stable/mps.html>
- <https://docs.pytorch.org/docs/stable/notes/mps.html>
- <https://developer.apple.com/documentation/metalperformanceshaders>

如果数据集太大而无法载入内存怎么办？我们可以使用一个名为 `mmap` 的 Unix 系统调用，它把磁盘上的文件映射到虚拟内存，并在访问该内存位置时才惰性加载文件内容。这样，你可以“假装”整个数据集都在内存里。NumPy 通过 `np.memmap` 实现了这一点（如果你当初是用 `np.save` 保存数组的，也可以用 `np.load` 的 `mmap_mode='r'` 标志），它会返回一个类 numpy 数组的对象，在你访问时按需加载条目。在训练期间从数据集（即 numpy 数组）采样时，务必以内存映射模式（通过 `np.memmap` 或 `np.load` 的 `mmap_mode='r'` 标志，取决于你保存数组的方式）承载数据集。同时务必指定与所加

载数组匹配的数据类型。显式验证内存映射的数据看起来正确（例如不包含超出预期词表大小的值）可能会有所帮助。

## 5.2 检查点（Checkpointing）

除了加载数据，我们还需要在训练过程中保存模型。运行作业时，我们常常希望能恢复一个中途停止的训练运行（例如由于作业超时、机器故障等）。即使一切顺利，我们之后可能也想拿到中间的模型（例如事后研究训练动态、从训练的不同阶段采样等）。

一个检查点应当包含恢复训练所需的全部状态。我们当然至少要能恢复模型权重。如果使用有状态的优化器（如 AdamW），我们还需要保存优化器的状态（例如 AdamW 的矩估计）。最后，要恢复学习率调度，我们需要知道停止时的迭代次数。PyTorch 让保存所有这些变得简单：每个 `nn.Module` 都有一个 `state_dict()` 方法，返回包含所有可学习权重的字典；之后我们可以用其姊妹方法 `load_state_dict()` 恢复这些权重。任何 `torch.optim.Optimizer` 也一样。最后，`torch.save(obj, dest)` 可以把一个对象（例如一个以张量为部分值的字典，也可以是整数等普通 Python 对象）转储到文件（路径）或类文件对象中，之后可用 `torch.load(src)` 载回内存。

### 问题（checkpointing）：实现模型检查点（1 分）

实现以下两个用于加载和保存检查点的函数：

`def save_checkpoint(model, optimizer, iteration, out)` 应当把来自模型、优化器和迭代次数的所有状态转储到类文件对象 `out` 中。你可以用模型和优化器的 `state_dict` 方法获取它们的相关状态，并用 `torch.save(obj, out)` 把 `obj` 转储到 `out`（PyTorch 同时支持路径和类文件对象）。常见的选择是让 `obj` 成为一个字典，但只要之后能加载你的检查点，用任何格式都可以。

该函数期望以下参数：

- `model: torch.nn.Module`
- `optimizer: torch.optim.Optimizer`
- `iteration: int`
- `out: str | os.PathLike | typing.BinaryIO | typing.IO[bytes]`

`def load_checkpoint(src, model, optimizer)` 应当从 `src`（路径或类文件对象）加载检查点，然后从该检查点恢复模型和优化器状态。你的函数应返回保存到检查点中的迭代次数。你可以用 `torch.load(src)` 恢复你在 `save_checkpoint` 实现中保存的内容，并用模型和优化器上的 `load_state_dict` 方法把它们恢复到先前的状态。

该函数期望以下参数：

- `src: str | os.PathLike | typing.BinaryIO | typing.IO[bytes]`
- `model: torch.nn.Module`

- `optimizer: torch.optim.Optimizer`

实现 `[adapters.run_save_checkpoint]` 和 `[adapters.run_load_checkpoint]` 适配器，并确保它们通过 `uv run pytest -k test_checkpointing`。

### 5.3 训练循环

现在，终于到了把你实现的全部组件整合进主训练脚本的时候。让启动不同超参数的训练运行变得容易（例如通过命令行参数传入）会有回报，因为之后你会多次这样做，以研究不同选择对训练的影响。

**问题 (`training_together`): 整合运行 (4 分)**

**交付内容:** 编写一个脚本，运行训练循环，在用户提供的输入上训练你的模型。特别地，我们建议你的训练脚本（至少）支持：

- 配置和控制各种模型与优化器超参数的能力。
- 用 `np.memmap` 省内存地加载大型训练集与验证集。
- 把检查点序列化到用户提供的路径。
- 定期记录训练和验证性能（例如输出到控制台和/或像 `Weights and Biases` 这样的外部服务）。<sup>a</sup>

---

<sup>a</sup>[wandb.ai](https://wandb.ai)

## 6 文本生成

现在我们可以训练模型了，最后一块拼图是从模型生成文本的能力。回顾一下，语言模型接收一个（可能批量的）长度为 `sequence_length` 的整数序列，产生一个大小为 `(sequence_length, vocab_size)` 的矩阵，其中序列的每个元素都是一个预测该位置之后下一个 token 的概率分布。现在我们来编写几个函数，把它变成新序列的采样方案。

### Softmax

按照标准约定，语言模型的输出是最后一个线性层的输出（“logits”），因此我们必须通过 softmax 运算（见前面的方程 10）把它转换为归一化概率。

### 解码 (Decoding)

要从模型生成（解码）文本，我们向模型提供一个前缀 token 序列（“提示” /prompt），让它产出词表上的概率分布以预测序列中的下一个 token。然后，我们从这个词表分布中采样，确定下一个输出 token。

具体来说，解码过程的一步应接收序列  $x_{1...t}$  并经由下式返回 token  $x_{t+1}$ ：

$$P(x_{t+1} = i \mid x_{1...t}) = \frac{\exp(v_i)}{\sum_j \exp(v_j)} \quad (21)$$

$$v_{1...t} = \text{TransformerLM}(x_t) \in \mathbb{R}^{\text{vocab\_size}} \quad (22)$$

其中 `TransformerLM` 是我们的模型，它接收长度为 `sequence_length` 的序列并产生大小为 `(sequence_length, vocab_size)` 的矩阵，我们取该矩阵的最后一个元素，因为我们要的是第  $t$  个位置的下一 token 预测。

通过反复从这些单步条件分布中采样（把先前生成的输出 token 追加到下一解码时间步的输入中），直到生成序列结束 token `<|endoftext|>`（或达到用户指定的最大生成 token 数），我们就得到了一个基本的解码器。

### 解码技巧

我们将使用小模型做实验，而小模型有时会生成质量很低的文本。两个简单的解码技巧可以帮助解决这些问题。第一，温度缩放（temperature scaling）：我们用温度参数  $\tau$  修改 softmax，新的 softmax 为

$$\text{softmax}(v, \tau)_i = \frac{\exp(v_i/\tau)}{\sum_{j=1}^{\text{vocab\_size}} \exp(v_j/\tau)}. \quad (23)$$

注意当  $\tau \rightarrow 0$  时， $v$  的最大元素会占据主导，softmax 的输出变成集中在这个最大元素上的 one-hot 向量。

第二，另一个技巧是核采样（nucleus / top-p sampling），即通过截断低概率 token 来修改采样分布。设  $q$  是我们由（温度缩放后的）softmax 得到的、大小为 `vocab_size` 的概率分布。带超参数  $p$  的核采样按下式产生下一个 token：

$$P(x_{t+1} = i \mid q) = \begin{cases} \frac{q_i}{\sum_{j \in V(p)} q_j} & \text{if } i \in V(p) \\ 0 & \text{otherwise} \end{cases} \quad (24)$$

其中  $V(p)$  是使  $\sum_{j \in V(p)} q_j \geq p$  成立的最小索引集合。你可以先按大小对概率分布  $q$  排序，然后选取最大的词表元素直到达到目标水平  $p$ ，从而轻松算出这个量。

**问题 (decoding): 解码 (3 分)**

**交付内容:** 实现一个从你的语言模型解码的函数。我们建议你支持以下特性:

- 为用户提供的提示生成补全 (即接收某个  $x_{1...t}$  并采样补全, 直到遇到 `<|endoftext|>` token)。
- 允许用户控制生成的最大 token 数。
- 给定期望的温度值, 在采样之前对预测的下一 token 分布应用 softmax 温度缩放。
- Top- $p$  采样 ([A. Holtzman et al., 2020], 也称为核采样), 使用用户指定的阈值。

## 7 实验

现在是把所有东西合在一起、在预训练数据集上训练 (小型) 语言模型的时候了。

### 7.1 如何运行实验与交付内容

理解 Transformer 架构组件背后原理的最好方式, 就是实际修改它并亲手运行。没有什么能替代动手实践。

为此, 能够快速、一致地做实验并记录你所做的事情非常重要。为了快速实验, 我们将在小规模模型 (总参数约 17M) 和简单数据集 (TinyStories) 上运行大量实验。为了保持一致性, 你将以系统化的方式消融组件、改变超参数; 为了记录, 我们会要求你提交实验日志以及与每个实验关联的学习曲线。

为了能够提交损失曲线, 请务必定期评估验证损失, 并同时记录步数和墙钟时间 (wall-clock time)。你可能会发现 Weights and Biases 之类的日志基础设施很有用。

**问题 (experiment\_log): 实验记录 (3 分)**

为你的训练和评估代码搭建实验跟踪基础设施, 使你能够对照梯度步数和墙钟时间跟踪你的实验与损失曲线。

**交付内容:** 你的实验日志基础设施代码, 以及本节下方作业问题的实验日志 (一份记录你所尝试过的所有事情的文档)。

### 7.2 TinyStories

我们将从一个非常简单的数据集 (TinyStories; R. Eldan et al. [1]) 开始, 模型会训练得很快, 我们也能观察到一些有趣的行为。获取该数据集的说明见第 1 节。下面是该数据集的一个样例。

**示例 (tinystories\_example): TinyStories 中的一条样例**

Once upon a time there was a little boy named Ben. Ben loved to explore the world around him. He saw many amazing things, like beautiful vases that were on display in a store. One day, Ben was walking through the store when he came across a very special vase. When Ben saw it he was amazed! He said, “Wow, that is a really amazing vase! Can I buy it?” The shopkeeper smiled and said, “Of course you can. You can take it home and show all your friends how amazing it is!” So Ben took the vase home and he was so proud of it! He called his friends over and showed them the amazing vase. All his friends thought the vase was beautiful and couldn’t believe how lucky Ben was. And that’s how Ben found an amazing vase in the store!

### 7.2.1 超参数调优

我们会告诉你一些起步用的非常基础的超参数，并请你为其余超参数找到一些效果良好的设置。

**词表大小 (Vocab size)** 10000。典型的词表大小在数万到数十万之间。你应当改变它，观察词表和模型行为如何变化。

**上下文长度 (Context length)** 256。像 TinyStories 这样的简单数据集可能不需要长序列，但对于稍后的 OpenWebText 数据，你可能想改变它。试着改变它，观察其对单次迭代运行时间和最终困惑度的影响。

**d\_model** 512。这比许多小型 Transformer 论文使用的 768 维略小，但会让事情更快。

**d\_ff** 1344。这约为  $\frac{8}{3}d_{\text{model}}$ ，同时是 64 的倍数，有利于 GPU 性能。

**RoPE theta 参数  $\Theta$**  10000。

**层数与头数** 4 层、16 头。合在一起会得到约 17M 非嵌入参数，是一个相当小的 Transformer。

**处理的总 token 数** 327,680,000 (你的批量大小  $\times$  总步数  $\times$  上下文长度应大致等于这个值)。

你应当反复试验，为以下其他超参数找到好的默认值：学习率、学习率预热、其他 AdamW 超参数 ( $\beta_1$ 、 $\beta_2$ 、 $\epsilon$ ) 以及权重衰减。你可以在 D. P. Kingma et al. [22] 中找到这类超参数的一些典型取值。

### 7.2.2 整合运行

现在你可以把一切整合起来：拿到训练好的 BPE 分词器，对训练数据集分词，然后在你编写的训练循环中运行。重要提示：如果你的实现正确且高效，上述超参数在 1 块 B200 GPU 上的运行时间应

当约为 20-30 分钟。如果你的运行时间长得多，请检查确保你的数据加载、检查点或验证损失代码没有成为运行时间的瓶颈，并确保你的实现正确进行了批处理。

### 7.2.3 调试模型架构的技巧与窍门

我们强烈建议你熟练使用 IDE 的内置调试器（例如 VSCode/Zed），比起用 `print` 语句调试，它会为你节省时间。如果你用的是文本编辑器，可以使用 `ipdb` 之类的工具。调试模型架构时还有其他一些好习惯：

- 开发任何神经网络架构时，一个常见的第一步是对单个 mini-batch 过拟合。如果你的实现正确，你应当能迅速把训练损失压到接近零。
- 在模型的各个组件中设置调试断点，检查中间张量的形状，确保它们符合你的预期。
- 监控激活值、模型权重和梯度的范数，确保它们没有爆炸或消失。

#### 问题 (`learning_rate`): 调整学习率 (2 B200 小时) (3 分)

学习率是最重要的待调超参数之一。以你训练的基础模型为出发点，回答以下问题：

(a) 对学习率做一次超参数扫描，报告最终损失（如果优化器发散，请注明发散）。

**交付内容：**与多个学习率关联的学习曲线。解释你的超参数搜索策略。

**交付内容：**一个在 TinyStories 上验证损失（每 token）不超过 1.45 的模型。

**低资源建议：在 CPU 或 Apple Silicon 上训练几步**

如果你在 `cpu` 或 `mps` 上运行，你应当把处理的总 token 数降到 40,000,000，这将足以生成相当流畅的文本。你也可以把目标验证损失从 1.45 放宽到 2.00。

在 M4 Max 芯片和 36 GB 内存上用调好学习率的我们的解答代码运行，我们使用批量大小  $\times$  总步数  $\times$  上下文长度 =  $32 \times 5000 \times 256 = 40,960,000$  个 token，在 `cpu` 上用时 1 小时 22 分钟，在 `mps` 上用时 36 分钟。在第 5000 步，我们取得 1.80 的验证损失。

另外几点提示：

- 当使用  $N$  个训练步时，我们建议调整余弦学习率衰减调度，使其恰好在第  $N$  步结束衰减（即达到最小学习率）。
- 当使用 `mps` 时，不要使用 TF32 内核，即不要像在 `cuda` 设备上那样设置 `torch.set_float32_matmul_precision('high')`。我们试着在 `mps`（torch 版本 2.9.0）上启用 TF32 内核，发现后端有时会静默地使用有缺陷的内核，导致训练不稳定。
- 你可以通过用 `torch.compile` 对模型进行 JIT 编译来加速训练。具体而言：
  - 在 `cpu` 上，用 `model = torch.compile(model)` 编译你的模型
  - 在 `mps` 上，可以用 `model = torch.compile(model, backend="aot_eager")` 在一定程度上优化反向传播

截至 torch 2.9.0，`mps` 不支持用 Inductor 编译。

(b) 民间经验认为最好的学习率在“稳定性的边缘”（at the edge of stability）。研究一下学习率开始发散的临界点与你最佳学习率之间的关系。

**交付内容：**递增学习率的学习曲线，其中至少包含一次发散的运行，并分析这与收敛速度的关系。

现在让我们改变批量大小，看看训练会发生什么。批量大小很重要——它让我们通过更大的矩阵乘法从 GPU 获得更高的效率，但批量大小总是越大越好吗？让我们运行一些实验来找出答案。

**问题（batch\_size\_experiment）：批量大小变化（1 B200 小时）（1 分）**

把你的批量大小从 1 一直变化到 GPU 显存上限。至少尝试中间的几个批量大小，包括 64 和 128 等典型值。

**交付内容：**不同批量大小运行的学习曲线。如有必要，应重新优化学习率。

**交付内容：**用几句话讨论你关于批量大小及其对训练影响的发现。

有了手上的解码器，我们现在可以生成文本了！我们将从模型生成文本，看看它有多好。作为参照，你得到的输出至少应当和下面的示例一样好。

**示例 (ts\_generate\_example): TinyStories 语言模型的采样输出**

Once upon a time, there was a pretty girl named Lily. She loved to eat gum, especially the big black one. One day, Lily's mom asked her to help cook dinner. Lily was so excited! She loved to help her mom. Lily's mom made a big pot of soup for dinner. Lily was so happy and said, "Thank you, Mommy! I love you." She helped her mom pour the soup into a big bowl. After dinner, Lily's mom made some yummy soup. Lily loved it! She said, "Thank you, Mommy! This soup is so yummy!" Her mom smiled and said, "I'm glad you like it, Lily." They finished cooking and continued to cook together. The end.

**低资源建议：在 CPU 或 Apple Silicon 上生成文本**

如果你改用了处理 40M token 的低资源配置，你看到的生成结果仍然像英语，但不会像上面那样流畅。例如，下面是我们用在一个在 40M token 上训练的 TinyStories 语言模型得到的采样输出：

Once upon a time, there was a little girl named Sue. Sue had a tooth that she loved very much. It was his best head. One day, Sue went for a walk and met a ladybug! They became good friends and played on the path together.  
"Hey, Polly! Let's go out!" said Tim. Sue looked at the sky and saw that it was difficult to find a way to dance shining. She smiled and agreed to help the talking!"  
As Sue watched the sky moved, what it was. She

下面是精确的问题描述和要求：

**问题 (generate): 生成文本 (1 分)**

使用你的解码器和训练好的检查点，报告你的模型生成的文本。你可能需要调整解码器参数（温度、top-p 等）以获得流畅的输出。

**交付内容：**至少 256 个 token 的文本转储（或直到第一个 `<|endoftext|>` token），并对该输出的流畅度做简短点评，以及至少两个影响输出好坏的因素。

## 7.3 消融实验与架构修改

理解 Transformer 的最好方式是实际修改它，观察它的行为。我们现在做一些简单的消融和修改。

### 消融 1：层归一化

人们常说层归一化对 Transformer 训练的稳定性很重要。但也许我们想活得刺激一点。让我们把 Transformer 块中的 RMSNorm 全部移除，看看会发生什么。

**问题 (layer\_norm\_ablation): 移除 RMSNorm 并训练 (0.5 B200 小时) (1 分)**

把 Transformer 中的所有 RMSNorm 移除并训练。在先前的最优学习率下会发生什么？用更低的学习率能获得稳定性吗？

**交付内容:** 移除 RMSNorm 并训练时的学习曲线，以及最优学习率下的学习曲线。

**交付内容:** 用几句话点评 RMSNorm 的影响。

现在让我们研究另一个乍看之下很随意的层归一化选择。Pre-norm Transformer 块定义为

$$z = x + \text{MultiHeadSelfAttention}(\text{RMSNorm}(x)) \quad (25)$$

$$y = z + \text{FFN}(\text{RMSNorm}(z)). \quad (26)$$

这是对原始 Transformer 架构为数不多的”共识性”修改之一；原始架构使用 post-norm 方式：

$$z = \text{RMSNorm}(x + \text{MultiHeadSelfAttention}(x)) \quad (27)$$

$$y = \text{RMSNorm}(z + \text{FFN}(z)). \quad (28)$$

让我们退回 post-norm 方式，看看会发生什么。

**问题 (pre\_norm\_ablation): 实现 post-norm 并训练 (0.5 B200 小时) (1 分)**

把你的 pre-norm Transformer 实现修改成 post-norm。用 post-norm 模型训练，看看会发生什么。

**交付内容:** post-norm Transformer 的学习曲线，并与 pre-norm 的进行比较。

我们看到，层归一化对 Transformer 的行为有重大影响，甚至连层归一化的位置都很重要。

## 消融 2: 位置嵌入

接下来我们将研究位置嵌入对模型性能的影响。具体来说，我们将比较我们的基础模型（带 RoPE）与完全不使用位置嵌入（NoPE）。事实证明，decoder-only Transformer（即像我们实现的这样带因果掩码的 Transformer）在理论上可以在不显式提供位置嵌入的情况下推断相对或绝对位置信息 [Y.-H. H. Tsai et al., 2019; A. Kazemnejad et al., 2023]。我们现在用实验检验 NoPE 与 RoPE 相比表现如何。

**问题 (no\_pos\_emb): 实现 NoPE (0.5 B200 小时) (1 分)**

修改你带 RoPE 的 Transformer 实现，把位置嵌入信息完全移除，看看会发生什么。

**交付内容:** 比较 RoPE 与 NoPE 性能的学习曲线。

### 消融 3: SwiGLU 与 SiLU

接下来，我们遵循 N. Shazeer [20]，通过比较 SwiGLU 前馈网络与使用 SiLU 激活但无门控线性单元 (GLU) 的前馈网络的性能，检验前馈网络中门控的重要性：

$$\text{FFN}_{\text{SiLU}}(x) = W_2 \text{SiLU}(W_1 x). \quad (29)$$

回顾一下，在我们的 SwiGLU 实现中，我们把前馈内层的维度设为约  $d_{\text{ff}} = \frac{8}{3}d_{\text{model}}$ （同时确保  $d_{\text{ff}} \bmod 64 = 0$ ，以利用 GPU 张量核心）。在这个消融基线中，你的 FFN 实现应改为设  $d_{\text{ffSiLU}} = 4 \times d_{\text{model}}$ ，以近似匹配默认 SwiGLU 前馈网络（它有三个而非两个权重矩阵）的参数数量。

**问题 (swiglu\_ablation): SwiGLU 与 SiLU (0.5 B200 小时) (1 分)**

**交付内容:** 比较参数量近似匹配的 SwiGLU 与 SiLU 前馈网络性能的学习曲线。

**交付内容:** 用几句话讨论你的发现。

**低资源建议: GPU 资源有限的线上学员应在 TinyStories 上测试修改**

在作业的剩余部分，我们将转向更大规模、更嘈杂的网页数据集 (OpenWebText)，试验架构修改并 (可选) 向课程排行榜提交成绩。

在 OpenWebText 上把语言模型训练到流畅需要很长时间，因此我们建议 GPU 资源有限的线上学员继续在 TinyStories 上测试修改 (用验证损失作为评估性能的指标)。

## 7.4 在 OpenWebText 上运行

我们现在转向一个更标准的、由网页爬取创建的预训练数据集。我们也提供了 OpenWebText [A. Gokaslan et al., 2019] 的小样本，保存为单个文本文件：如何获取见第 1 节。

下面是 OpenWebText 中的一条样例。注意文本要真实、复杂和多样得多。你可能想翻一翻训练数据集，感受一下网页爬取语料库的训练数据长什么样。

**示例 (owt\_example): OWT 中的一条样例**

Baseball Prospectus director of technology Harry Pavlidis took a risk when he hired Jonathan Judge.

Pavlidis knew that, as Alan Schwarz wrote in *The Numbers Game*, “no corner of American culture is more precisely counted, more passionately quantified, than performances of baseball players.” With a few clicks here and there, you can find out that Noah Syndergaard’s fastball revolves more than 2,100 times per minute on its way to the plate, that Nelson Cruz had the game’s highest average exit velocity among qualified hitters in 2016 and myriad other tidbits that seem ripped from a video game or science fiction novel. The rising ocean of data has empowered an increasingly important actor in baseball’s culture: the analytical hobbyist.

That empowerment comes with added scrutiny – on the measurements, but also on the people and publications behind them. With Baseball Prospectus, Pavlidis knew all about the backlash that accompanies quantitative imperfection. He also knew the site’s catching metrics needed to be reworked, and that it would take a learned mind – someone who could tackle complex statistical modeling problems – to complete the job.

“He freaks us out.” —Harry Pavlidis

Pavlidis had a hunch that Judge “got it” based on the latter’s writing and their interaction at a site-sponsored ballpark event. [...]

**注意：**在此实验中，你可能需要重新调整学习率或批量大小等超参数。

**问题 (main\_experiment): 在 OWT 上实验 (2 B200 小时) (2 分)**

在 OpenWebText 上，用与 TinyStories 相同的模型架构和总训练迭代次数训练你的语言模型。这个模型表现如何？

**交付内容：**你的语言模型在 OpenWebText 上的学习曲线。描述与 TinyStories 相比的损失差异——我们该如何解读这些损失？

**交付内容：**OpenWebText 语言模型生成的文本，格式与 TinyStories 输出相同。这段文本的流畅度如何？既然我们使用了与 TinyStories 相同的模型和算力预算，为什么输出质量更差？

## 7.5 你自己的改进 + 排行榜

恭喜你走到这一步。你快完成了！现在你将尝试改进 Transformer 架构，看看你的超参数和架构与班上其他同学相比表现如何。

**排行榜规则**

除以下规定外没有任何限制：

- **运行时间：**你的提交在 B200 上最多运行 45 分钟。如果你使用 SLURM 或 Modal，可能需要在提交脚本中强制执行这一点。
- **数据：**你只能使用我们提供的 OpenWebText 训练数据集。

除此之外，尽管随心所欲。

如果你想找一些可实现的点子，可以看看这些资源：

- 最先进的开源 LLM 系列，例如 Llama 3 [A. Grattafiori et al., 2024] 或 Qwen 2.5 [A. Yang et al., 2024]。
- NanoGPT speedrun 仓库 ([github.com/KellerJordan/modded-nanogpt](https://github.com/KellerJordan/modded-nanogpt))，社区成员在其中发布了许多用于“速通”小规模语言模型预训练的有趣修改。例如，一个可以追溯到原始 Transformer 论文的常见修改是把输入嵌入和输出嵌入的权重绑定在一起（见 A. Vaswani et al. [8]（3.4 节）和 A. Chowdhery et al. [16]（2 节））。如果你尝试权重绑定（weight tying），可能需要降低嵌入/LM head 初始化的标准差。

在尝试完整的 45 分钟运行之前，你应当先在 OpenWebText 的小子集或 TinyStories 上测试这些想法。

需要提醒的是，你可能会发现在这个排行榜上表现良好的某些修改未必能推广到更大规模的预训练。我们将在课程的缩放定律（scaling laws）单元进一步探讨这个想法。

**问题（leaderboard）：排行榜（10 B200 小时）（6 分）**

你将在上述排行榜规则下训练一个模型，目标是在 0.75 B200 小时内最小化你语言模型的验证损失。

**交付内容：**记录到的最终验证损失、一条清晰展示以墙钟时间为横轴（小于 45 分钟）的学习曲线，以及你所做工作的描述。我们期望排行榜提交至少击败 5.0 损失的朴素基线。在此提交排行榜：[github.com/stanford-cs336/assignment1-basics-leaderboard](https://github.com/stanford-cs336/assignment1-basics-leaderboard)。

## 参考文献

（参考文献保留英文原文，以便查阅。）

- [1] R. Eldan and Y. Li, “TinyStories: How Small Can Language Models Be and Still Speak Coherent English?.” 2023.
- [2] A. Gokaslan, V. Cohen, E. Pavlick, and S. Tellex, “OpenWebText corpus.” 2019.

- [3] R. Sennrich, B. Haddow, and A. Birch, “Neural Machine Translation of Rare Words with Subword Units,” in *Proc. of ACL*, 2016.
- [4] C. Wang, K. Cho, and J. Gu, “Neural Machine Translation with Byte-Level Subwords.” 2019.
- [5] P. Gage, “A new algorithm for data compression,” *C Users Journal*, vol. 12, no. 2, pp. 23–38, Feb. 1994.
- [6] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, “Language Models are Unsupervised Multitask Learners.” 2019.
- [7] A. Radford, K. Narasimhan, T. Salimans, and I. Sutskever, “Improving Language Understanding by Generative Pre-Training.” 2018.
- [8] A. Vaswani et al., “Attention is All you Need,” in *Proc. of NeurIPS*, 2017.
- [9] T. Q. Nguyen and J. Salazar, “Transformers without Tears: Improving the Normalization of Self-Attention,” in *Proc. of IWSWLT*, 2019.
- [10] R. Xiong et al., “On Layer Normalization in the Transformer Architecture,” in *Proc. of ICML*, 2020.
- [11] J. L. Ba, J. R. Kiros, and G. E. Hinton, “Layer Normalization.” 2016.
- [12] H. Touvron et al., “LLaMA: Open and Efficient Foundation Language Models.” 2023.
- [13] B. Zhang and R. Sennrich, “Root Mean Square Layer Normalization,” in *Proc. of NeurIPS*, 2019.
- [14] A. Grattafiori et al., “The Llama 3 Herd of Models.” [Online]. Available: <https://arxiv.org/abs/2407.21783>
- [15] A. Yang et al., “Qwen2.5 Technical Report,” *arXiv preprint* arXiv:2412.15115, 2024.
- [16] A. Chowdhery et al., “PaLM: Scaling Language Modeling with Pathways.” 2022.
- [17] D. Hendrycks and K. Gimpel, “Bridging Nonlinearities and Stochastic Regularizers with Gaussian Error Linear Units.” 2016.
- [18] S. Elfving, E. Uchibe, and K. Doya, “Sigmoid-Weighted Linear Units for Neural Network Function Approximation in Reinforcement Learning.” [Online]. Available: <https://arxiv.org/abs/1702.03118>
- [19] Y. N. Dauphin, A. Fan, M. Auli, and D. Grangier, “Language Modeling with Gated Convolutional Networks.” [Online]. Available: <https://arxiv.org/abs/1612.08083>

- [20] N. Shazeer, “GLU Variants Improve Transformer.” 2020.
- [21] J. Su, Y. Lu, S. Pan, B. Wen, and Y. Liu, “RoFormer: Enhanced Transformer with Rotary Position Embedding.” 2021.
- [22] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” in *Proc. of ICLR*, 2015.
- [23] I. Loshchilov and F. Hutter, “Decoupled Weight Decay Regularization,” in *Proc. of ICLR*, 2019.
- [24] T. B. Brown et al., “Language Models are Few-Shot Learners,” in *Proc. of NeurIPS*, 2020.
- [25] J. Kaplan et al., “Scaling Laws for Neural Language Models.” 2020.
- [26] J. Hoffmann et al., “Training Compute-Optimal Large Language Models.” 2022.
- [27] A. Holtzman, J. Buys, L. Du, M. Forbes, and Y. Choi, “The Curious Case of Neural Text Degeneration,” in *Proc. of ICLR*, 2020.
- [28] Y.-H. H. Tsai, S. Bai, M. Yamada, L.-P. Morency, and R. Salakhutdinov, “Transformer Dissection: An Unified Understanding for Transformer’s Attention via the Lens of Kernel,” in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, K. Inui, J. Jiang, V. Ng, and X. Wan, Eds., Hong Kong, China: Association for Computational Linguistics, Nov. 2019, pp. 4344–4353. doi: [10.18653/v1/D19-1443](https://doi.org/10.18653/v1/D19-1443).
- [29] A. Kazemnejad, I. Padhi, K. Natesan, P. Das, and S. Reddy, “The Impact of Positional Encoding on Length Generalization in Transformers,” in *Thirty-seventh Conference on Neural Information Processing Systems*, 2023. [Online]. Available: <https://openreview.net/forum?id=Dr1l2gcjzl>